

Vue 3 – The Complete Guide Course

E-BOOK



www.dotnetmastery.com

Bhruugen Patel

INDEX

<u>CREATE VUE PROJECT</u>	<u>6</u>
<u>HELLO VUE</u>	<u>7</u>
<u>RETURNING MULTIPLE PROPERTIES</u>	<u>8</u>
<u>OBJECT AND ARRAYS</u>	<u>9</u>
<u>FUNCTIONS IN VUE</u>	<u>10</u>
<u>PROJECT: REF</u>	<u>11</u>
<u>REF ON OBJECT</u>	<u>12</u>
<u>REACTIVE IN VUE</u>	<u>13</u>
<u>PROJECT: REACTIVE AND NON-REACTIVITY IN VUE</u>	<u>14</u>
<u>PROJECT: V-HTML DIRECTIVE</u>	<u>15</u>
<u>PROJECT: V-ONCE DIRECTIVE</u>	<u>16</u>
<u>PROJECT: V-BIND DIRECTIVE</u>	<u>17</u>
<u>PROJECT: V-SHOW DIRECTIVE</u>	<u>18</u>
<u>PROJECT: V-IF DIRECTIVE</u>	<u>19</u>
<u>PROJECT: V-ELSE-IF AND V-ELSE VUE DIRECTIVE</u>	<u>20</u>
<u>PROJECT: V-FOR DIRECTIVE</u>	<u>21</u>
<u>PROJECT: ITERATE OVER ARRAY OF OBJECTS</u>	<u>22</u>
<u>PROJECT: V-ONCE DIRECTIVE INSIDE LOOP</u>	<u>23</u>
<u>PROJECT: KEY IN V-FOR DIRECTIVE</u>	<u>24</u>
<u>PROJECT: DYNAMIC STYLE BINDINGS</u>	<u>25</u>
<u>PROJECT: CLASS BINDINGS IN VUE</u>	<u>26</u>

PROJECT: ARRAY OF CLASSES AND CUSTOM STYLE OBJECT	27
PROJECT: BUTTON ON CLICK EVENT	29
PROJECT: PASSING PARAMETERS TO FUNCTIONS	30
PROJECT: EVENT OBJECT	31
PROJECT: EVENT MODIFIERS	32
PROJECT: TWO-WAY DATA BINDING	33
PROJECT: MODEL MODIFIERS	34
PROJECT: COMPUTED PROPERTIES	35
PROJECT: WATCHERS IN VUE	36
PROJECT: WATCHERS CAN BE ASYNCHRONOUS	37
PROJECT: SHORTCUT FOR V-ON	38
PROJECT: FORMS SETUP	39
PROJECT: PREVENT SUBMIT IN FORMS	40
PROJECT: FORMS VALIDATION	42
PROJECT: CREATE FIRST COMPONENT	44
PROJECT: CLEANER SETUP	45
PROJECT: LOCAL VS GLOBAL COMPONENTS	46
PROJECT: PASSING PROPS	48
PROJECT: BINDED PROPS	49
PROJECT: VALIDATING PROPS	50
PROJECT: PROPS AND LOCAL VARIABLE	51
PROJECT: PASSING DYNAMIC PROP BINDING	52
PROJECT: DYNAMICALLY CHANGE CLASS AND BUTTON TEXT	54

PROJECT: EMITS IN ACTION	56
PROJECT: PASSING PARAMETERS IN IN EMITS	58
PROJECT: PASSING OBJECTS IN EMIT	60
PROJECT: ADD NEW CONTACT COMPONENT	62
PROJECT: PASSING PROPS DOWN TO GRANDCHILD COMPONENT	65
PROJECT: PROVIDE AND INJECT	68
PROJECT: DYNAMIC COMPONENT	71
PROJECT: KEEP ALIVE	73
PROJECT: SLOTS	75
PROJECT: NAMED SLOTS	77
PROJECT: SCOPED STYLES	80
PROJECT: LIFECYCLE DEMO	81
PROJECT: COMPOSABLE IN VUE	83
PROJECT: PROPS IN COMPOSABLE	84
PROJECT: ROUTING IN ACTION	85
PROJECT: ROUTER LINK	87
PROJECT: DYNAMIC ROUTE BINDING	88
PROJECT: MULTIPLE PARAMETERS IN URL:	89
PROJECT: ROUTE WITH DYNAMIC VALUES	90
PROJECT: NAMED ROUTES	91
PROJECT: REDIRECT URL	92
PROJECT: NOT FOUND	93
PROJECT: NAVIGATE USING SCRIPT	94

PROJECT: BACK BUTTON	95
PROJECT: PROTECTED PAGES FROM UNAUTHENTICATED ACCESS	96
PROJECT: PER ROUTE NAVIGATION GUARD	97
PROJECT: AXIOS IN ACTION	99
PROJECT: FETCH IN ACTION	100
PROJECT: PINIA	101
PROJECT: PINIA ACTIONS AND GETTERS	102

Create Vue Project

A Vue 3 CDN project is a simple and efficient way to start building web applications without the need for a complex setup or local installations. Using a CDN link, you can directly include Vue.js in your HTML file.

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div>Hello</div>
  </body>
</html>
```

Output:



Hello Vue

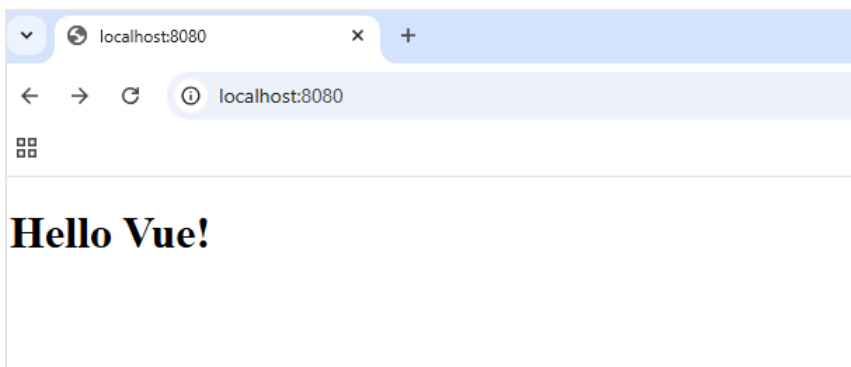
A "Hello Vue" app is the simplest Vue.js project for beginners. It typically displays a simple "Hello, Vue!" message on the screen and demonstrates core Vue features like template binding.

This app initiates a Vue instance and binds data to the view. We are setting message with a string and accessing that in the HTML using special `{{ }}` which renders Vue code.

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app">
      <h1>{{message}}</h1>
    </div>
  </body>
  <script>
    const { createApp } = Vue;
    createApp({
      setup() {
        return {
          message: "Hello Vue!",
        };
      },
    }).mount("#app");
  </script>
</html>
```

Output:



Returning multiple properties

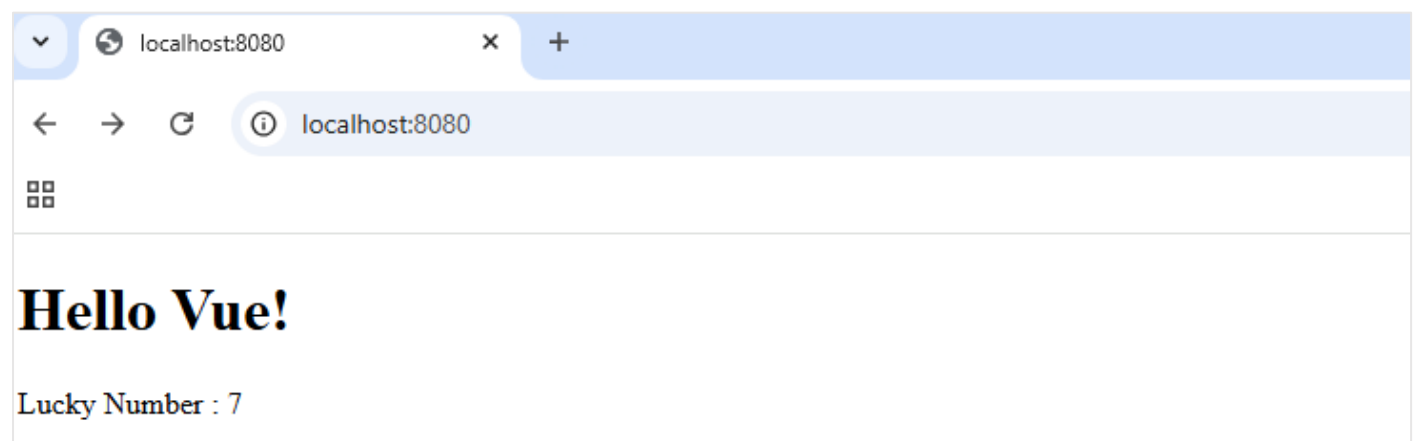
A Vue app can return multiple properties from its setup functions to demonstrate Vue's ability to manage and display reactive data. Each property can be dynamically bound to the template.

This example returns and displays multiple properties (**message** and **luckyNumber**) from the data object, highlighting Vue's powerful data binding.

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app">
      <h1>{{message}}</h1>
      <p>Lucky Number : {{luckyNumber}}</p>
    </div>
  </body>
  <script>
    const { createApp } = Vue;
    createApp({
      setup() {
        const message = "Hello Vue!";
        const luckyNumber = 7;
        return {
          message,
          luckyNumber,
        };
      },
    }).mount("#app");
  </script>
</html>
```

Output:



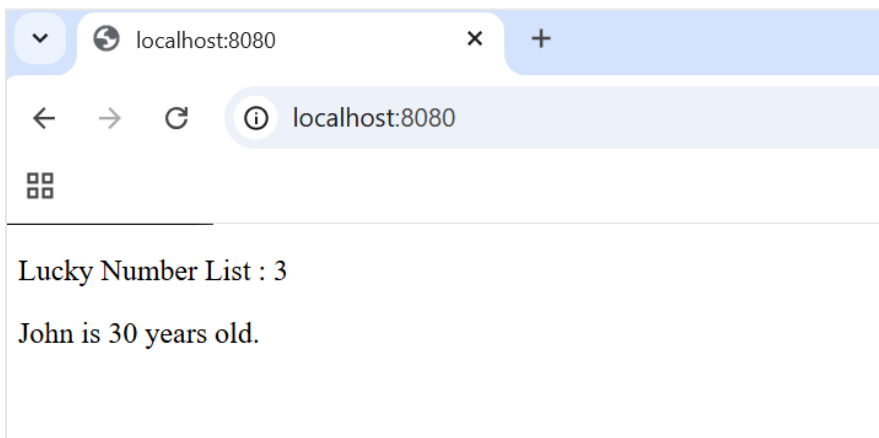
Object and Arrays

This example demonstrates how Vue can manage various data types such as strings, numbers, arrays, and objects efficiently, and bind them dynamically to the DOM. It is a nice introduction to working with multiple data structures in Vue.

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app">
      <p>Lucky Number List : {{luckyNumbers[0]}}</p>
      <p>{{employee.name}} is {{employee.age}} years old.</p>
    </div>
  </body>
  <script>
    const { createApp } = Vue;
    createApp({
      setup() {
        const luckyNumbers = [3, 7, 9, 11]; // array
        const employee = { // object
          name: "John",
          age: 30,
          salary: 50000,
        };
        return {
          luckyNumbers,
          employee,
        };
      },
    }).mount("#app");
  </script>
</html>
```

Output:



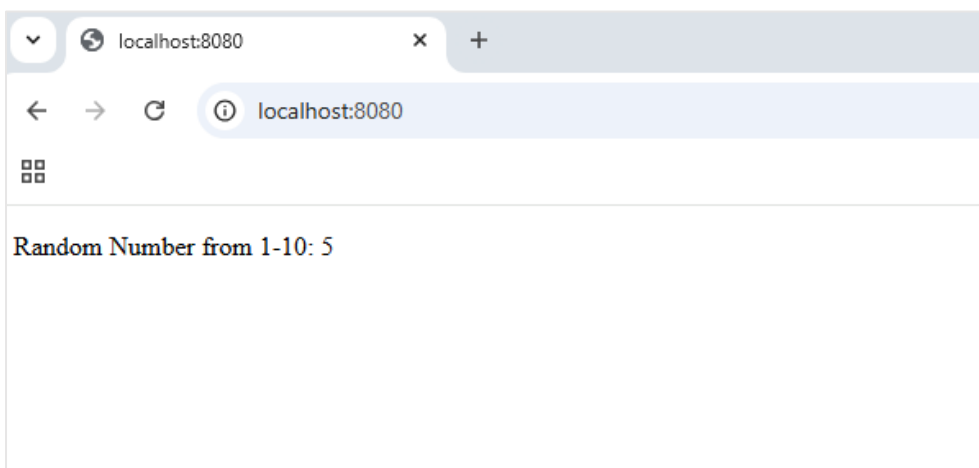
Functions in Vue

This example demonstrates Vue's reactivity, template expressions, and integration with custom JavaScript functions for interactivity

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app">
      <p>Random Number from 1-10: {{ getRandomLuckyNumber() }}</p>
    </div>
  </body>
  <script>
    const { createApp } = Vue;
    createApp({
      setup() {
        function getRandomLuckyNumber() { //function in Vue
          return Math.floor(Math.random() * 10) + 1;
        }
        return {
          getRandomLuckyNumber,
        };
      },
    }).mount("#app");
  </script>
</html>
```

Output:



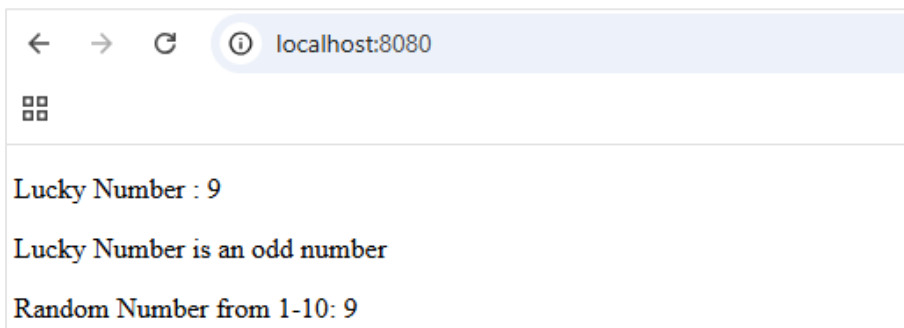
Project: Ref

In Vue 3, the `ref` function can be utilized to bind a value, ensuring that any updates to the value are automatically reflected in the DOM.

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app">
      <p>Lucky Number : {{luckyNumber}}</p>
      <p>
        Lucky Number is an {{luckyNumber%2 == 0 ? "even number":"odd number"}}
      </p>
      <p>Random Number from 1-10: {{ getRandomLuckyNumber() }}</p>
    </div>
  </body>
  <script>
    const { createApp, ref } = Vue;
    createApp({
      setup() {
        const luckyNumber = ref(6);
        function getRandomLuckyNumber() {
          luckyNumber.value = Math.floor(Math.random() * 10) + 1;
          return Math.floor(Math.random() * 10) + 1;
        }
        return {
          luckyNumber,
          getRandomLuckyNumber,
        };
      },
    }).mount("#app");
  </script>
</html>
```

Output:

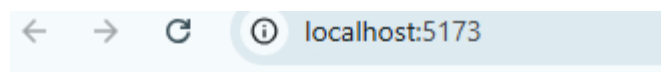


Ref on Object

Example of reactivity using ref and dynamic rendering on object.

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app">
      <p>{{employee.name}} is {{employee.age}} years old.</p>
      {{updateEmployeeSalary()}}
      <p>His salary is {{employee.salary}}</p>
    </div>
  </body>
  <script>
    const { createApp, ref } = Vue;
    createApp({
      setup() {
        const employee = ref({
          name: "John",
          age: 30,
          salary: 50000,
        });
        function updateEmployeeSalary() {
          setTimeout(() => {
            employee.value.salary = 60000;
          }, 2000);
        }
        return {
          employee,
          updateEmployeeSalary,
        };
      },
    }).mount("#app");
  </script>
</html>
```



John is 30 years old.

His salary is 50000

Reactive in Vue

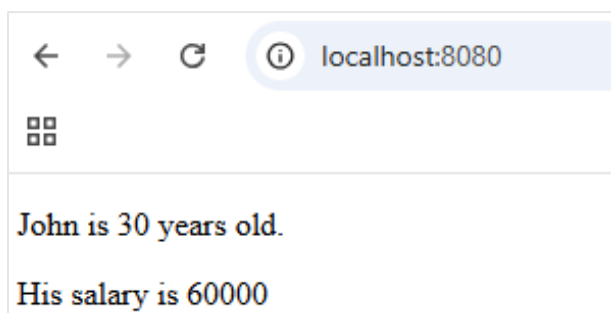
In Vue 3, `reactive()` is used to create a reactive state object. Unlike `ref`, which is for single primitive values, `reactive()` works best for creating reactive objects or arrays where all nested properties can be tracked and updated.

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app">
      <p>{{ updateEmployeeSalary() }}</p>
      <p>{{employee.name}} is {{employee.age}} years old.</p>
      <p>His salary is {{employee.salary}}</p>
    </div>
  </body>
  <script>
    const { createApp, reactive } = Vue;
    createApp({
      setup() {
        const employee = reactive({
          name: "John",
          age: 30,
          salary: 50000,
        });
        function updateEmployeeSalary() {
          setTimeout(() => {
            employee.salary = 60000;
          }, 2000);
        }

        return {
          employee,
          updateEmployeeSalary,
        };
      },
    }).mount("#app");
  </script>
</html>
```

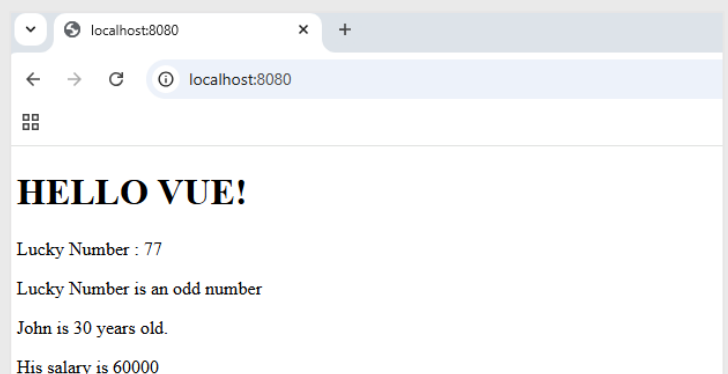
Output:



Project: Reactive and Non-Reactivity in Vue

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app">
      <h1>{{getMessage()}}</h1>
      <p>Lucky Number : {{luckyNumber}}</p>
      <p>
        Lucky Number is an {{luckyNumber%2 == 0 ? "even number":"odd number"}}
      </p>
      <p>{{ updateLuckNumber() }}</p>
      <p>{{employee.name}} is {{employee.age}} years old.</p>
      <p>His salary is {{employee.salary}}</p>
    </div>
  </body>
</html>
<script>
const { createApp, ref, reactive } = Vue;
createApp({
  setup() {
    const message = "Hello Vue!"; // Non Reactive
    const luckyNumber = ref(6); // Ref
    const employee = reactive({ // Reactive
      name: "John",
      age: 30,
      salary: 50000,
    });
    function updateLuckNumber() {
      setTimeout(() => {
        luckyNumber.value = 77;
        employee.salary = 60000;
      }, 2000);
    }
    function getMessage() {
      return message.toUpperCase();
    }
    return {
      luckyNumber,
      employee,
      updateLuckNumber,
      getMessage,
    };
  },
}).mount("#app");
</script>
</html>
```



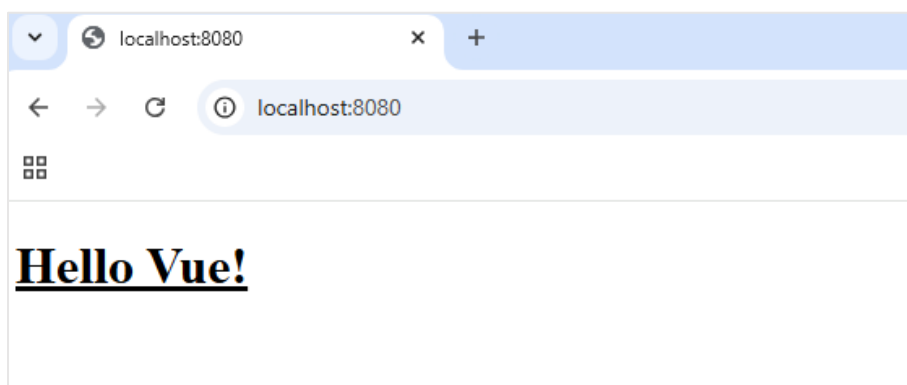
Project: v-html directive

The v-html directive in Vue.js is used to render raw HTML content within an element. It is particularly useful when you need to dynamically display HTML content.

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app">
      <span v-html="getMessage()"></span>
    </div>
  </body>
  <script>
    const { createApp } = Vue;
    createApp({
      setup() {
        const message = "Hello Vue!";
        function getMessage() {
          return `<h1><u>${message}</u></h1>`;
        }
        return {
          getMessage,
        };
      },
    }).mount("#app");
  </script>
</html>
```

Output:



Project: v-once directive

The v-once directive in Vue.js renders an element or component a single time. After the initial render, Vue will skip future updates for that element, making it helpful for static content that does not change.

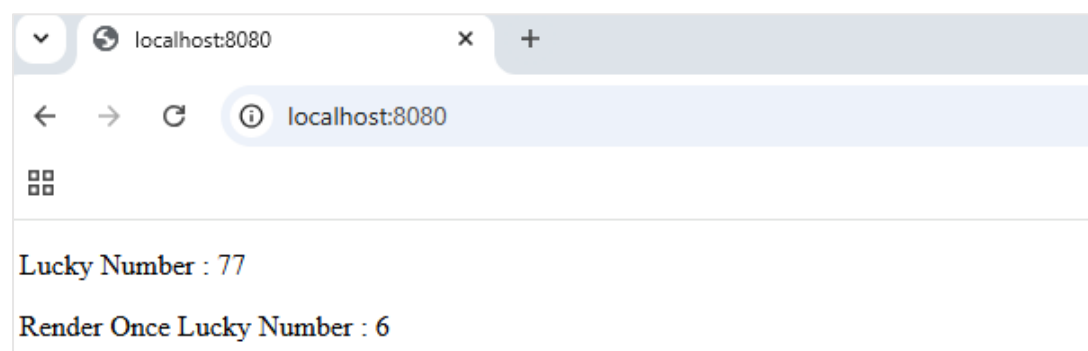
In this code, you can see Render Once Lucky Number will not be modified even if luckyNumber value gets updated since we have v-once.

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app">
      <p>Lucky Number : {{luckyNumber}}</p>
      <p v-once>Render Once Lucky Number : {{luckyNumber}}</p>

      <p>{{ updateLuckyNumber() }}</p>
    </div>
  </body>
  <script>
    const { createApp, ref, reactive } = Vue;
    createApp({
      setup() {
        const message = "Hello Vue!";
        const luckyNumber = ref(6);
        function updateLuckyNumber() {
          setTimeout(() => {
            luckyNumber.value = 77;
          }, 2000);
        }
        return {
          luckyNumber,
          updateLuckyNumber,
        };
      },
    }).mount("#app");
  </script>
</html>
```

Output:



Project: v-bind directive

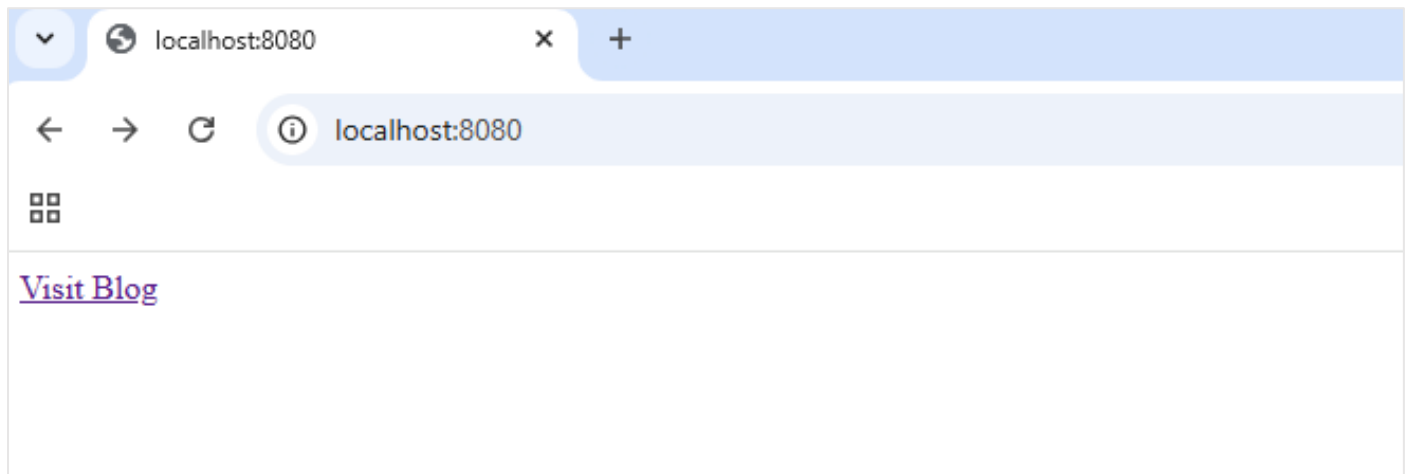
The v-bind directive in Vue.js is used to dynamically bind an HTML attribute to a JavaScript expression.

The v-bind directive is perfect for binding dynamic data to element attributes like src, href, class, or id.

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app">
      <a v-bind:href="blogUrl">Visit Blog</a>
    </div>
  </body>
  <script>
    const { createApp } = Vue;
    createApp({
      setup() {
        const blogUrl = "https://www.dotnetmastery.com";
        return {
          blogUrl,
        };
      },
    }).mount("#app");
  </script>
</html>
```

Output:



Project: v-show directive

The v-show directive in Vue.js is used to conditionally display an element by toggling its display CSS property. It simply hides or shows it based on the provided condition.

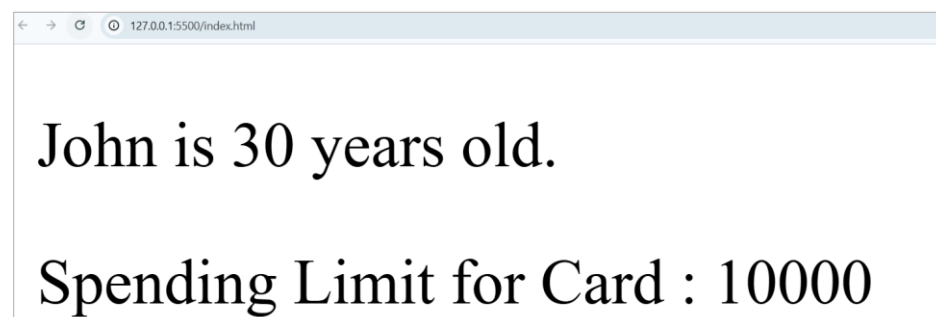
Use v-show for toggling visibility when you need the element to remain in the DOM for performance reasons, as it avoids the re-rendering cost of v-if.

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app">
      <p>{{employee.name}} is {{employee.age}} years old.</p>

      <p v-show="employee.hasCorporateCard">
        Spending Limit for Card : {{employee.spendingLimit}}
      </p>
    </div>
  </body>
  <script>
    const { createApp, reactive } = Vue;
    createApp({
      setup() {
        const employee = reactive({
          name: "John",
          age: 30,
          hasCorporateCard: true,
          spendingLimit: 10000,
        });
        return {
          employee,
        };
      },
    }).mount("#app");
  </script>
</html>
```

Output:



Project: v-if directive

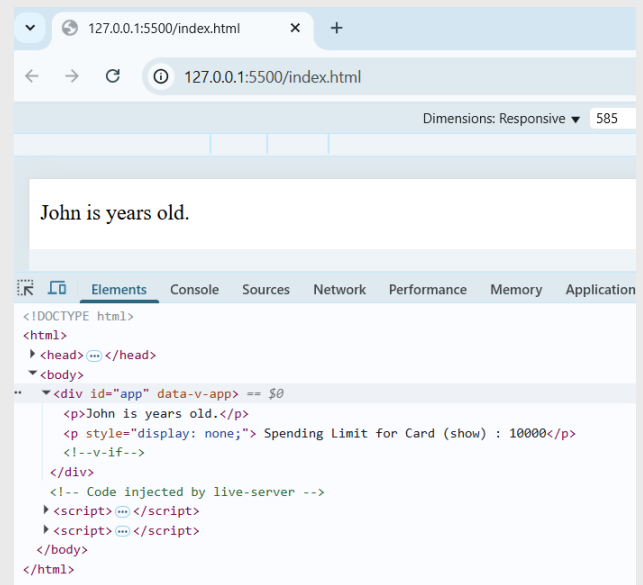
The v-if directive in Vue.js is used to conditionally render elements. If the condition evaluates to false, the element is completely removed from the DOM.

This can help optimize performance when rendering substantial amounts of data.

Use v-if for scenarios where rendering costs matter and the condition changes infrequently. Use v-show for faster toggling when the element needs to stay in the DOM. If condition is false for v-show then also it will be in DOM but with display none.

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app">
      <p>{{employee.name}} is {{employee.age}} years old.</p>
      <p v-show="employee.hasCorporateCard">
        Spending Limit for Card (show) : {{employee.spendingLimit}}
      </p>
      <p v-if="employee.hasCorporateCard">
        Spending Limit for Card (if) : {{employee.spendingLimit}}
      </p>
    </div>
  </body>
</html>
<script>
  const { createApp, reactive } = Vue;
  createApp({
    setup() {
      const employee = reactive({
        name: "John",
        hasCorporateCard: false,
        spendingLimit: 10000,
      });
      return {
        employee,
      };
    },
  }).mount("#app");
</script>
</html>
```

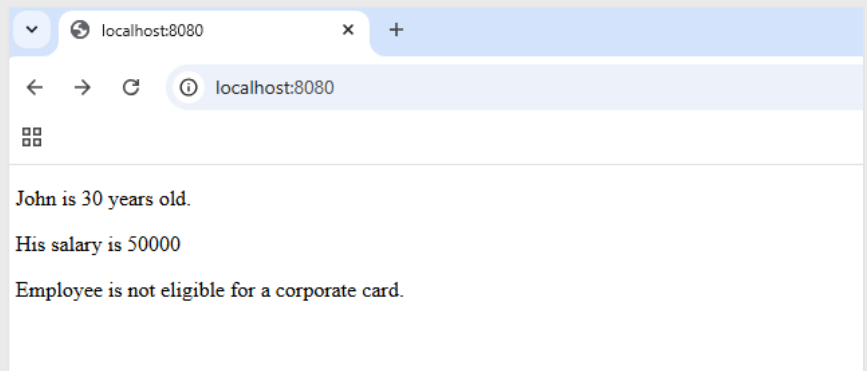


Project: v-else-if and v-else Vue directive

v-else-if and v-else are great to manage conditional flows with further conditions in Vue templates

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app">
      <p>{{employee.name}} is {{employee.age}} years old.</p>
      <p>His salary is {{employee.salary}}</p>
      <p v-if="employee.hasCorporateCard">
        Spending Limit for Card (if) : {{employee.spendingLimit}}
      </p>
      <p v-else-if="employee.salary > 25000 && employee.yearsWithCompany > 1">
        Employee is eligible for a corporate card.
      </p>
      <p v-else>Employee is not eligible for a corporate card.</p>
    </div>
  </body>
  <script>
    const { createApp, reactive } = Vue;
    createApp({
      setup() {
        const employee = reactive({
          name: "John",
          age: 30,
          salary: 50000,
          hasCorporateCard: false,
          spendingLimit: 10000,
          yearsWithCompany: 0,
        });
        return {
          employee,
        };
      },
    }).mount("#app");
  </script>
</html>
```



Project: v-for directive

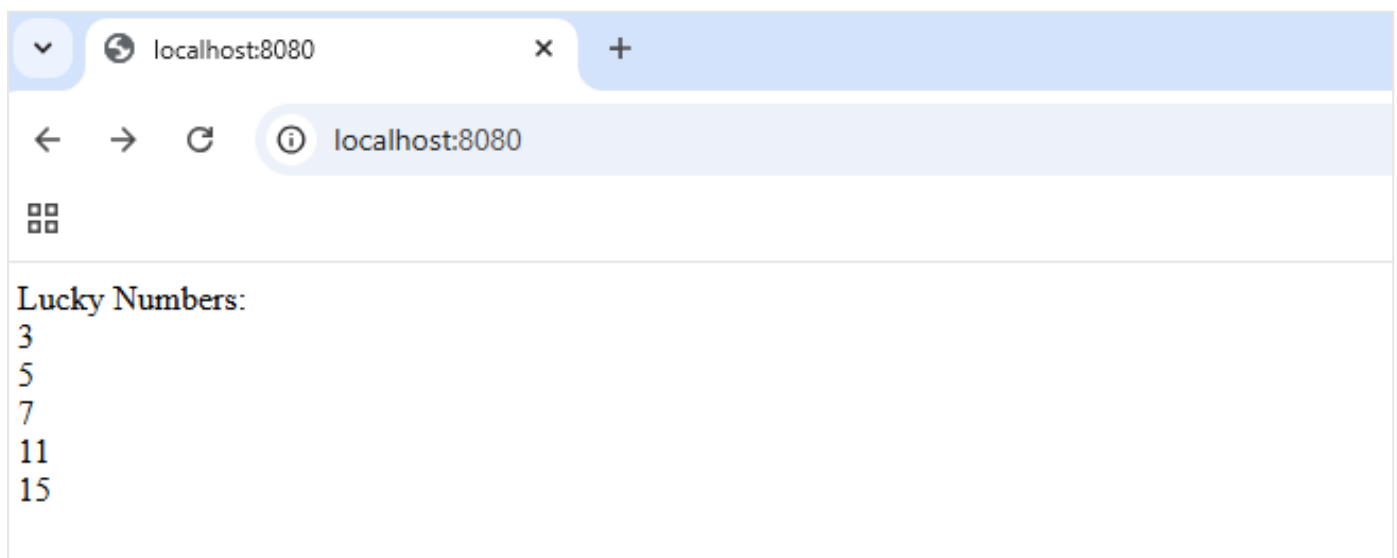
The v-for directive in Vue.js is used to render a list of items by iterating over an array or object. It allows you to dynamically create elements based on the data provided.

It is used for rendering lists, tables, or repeating structures in your Vue application.

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app">
      Lucky Numbers:
      <span v-for="(luckyNumber, index) in luckyNumbers">
        <br />
        {{luckyNumber}}
      </span>
    </div>
  </body>
  <script>
    const { createApp } = Vue;
    createApp({
      setup() {
        const luckyNumbers = [3, 5, 7, 11, 15];
        return {
          luckyNumbers,
        };
      },
    }).mount("#app");
  </script>
</html>
```

Output:



Project: Iterate over Array of Objects

The v-for directive in Vue.js is commonly used to iterate over an array of objects and render their properties dynamically.

Use v-for="item in items" to iterate over the array, and item.property to access individual object properties.

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app">
      <p>Projects in Pipeline</p>
      <ul>
        <li v-for="(project, index) in employee.projects">
          <span>
            {{index + 1}} - "{{project.project}}" timeline is -
            {{project.timeline}}
          </span>
        </li>
      </ul>
    </div>
  </body>
  <script>
    const { createApp, reactive } = Vue;
    createApp({
      setup() {
        const employee = reactive({
          name: "John",
          age: 30,
          salary: 50000,
          hasCorporateCard: false,
          spendingLimit: 10000,
          yearsWithCompany: 0,
          projects: [
            { project: "Green A", timeline: "2 years" },
            { project: "Blue H", timeline: "9 months" },
            { project: "Pink G", timeline: "3 years" },
          ],
        });
        return {
          employee,
        };
      },
    }).mount("#app");
  </script>
</html>
```

Projects in Pipeline

- 1 - "Green A" timeline is - 2 years
- 2 - "Blue H" timeline is - 9 months
- 3 - "Pink G" timeline is - 3 years

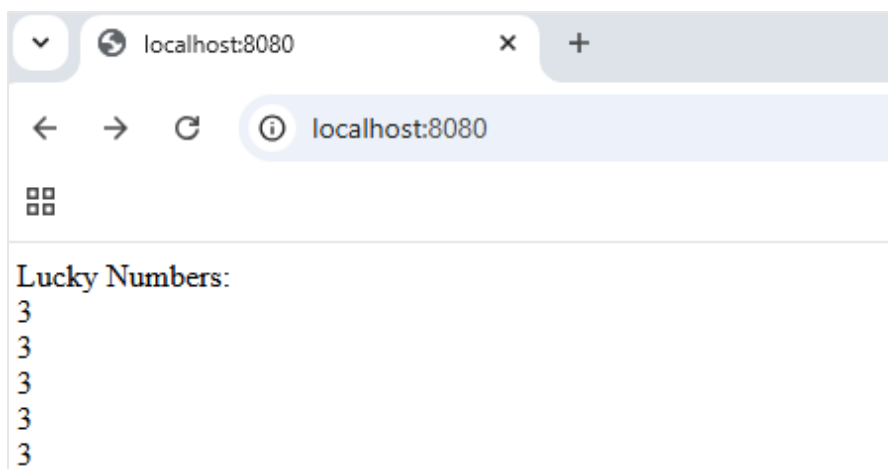
Project: v-once directive inside loop

The v-once directive can be used inside a v-for loop to render a static version of an element during the first render. This is helpful when you don't want items within the loop to re-render on subsequent updates.

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app">
      Lucky Numbers:
      <span v-for="(luckyNumber, index) in luckyNumbers">
        <br />
        <span v-once>{{luckyNumber}}</span>
      </span>
      <hr />
    </div>
  </body>
  <script>
    const { createApp, ref, reactive } = Vue;
    createApp({
      setup() {
        const luckyNumbers = [3, 5, 7, 11, 15];
        return {
          luckyNumbers,
        };
      },
    }).mount("#app");
  </script>
</html>
```

Output:

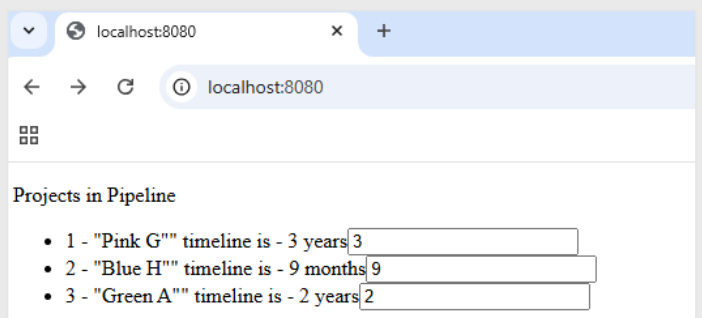


Project: key in v-for directive

The key attribute in a v-for loop is critical in Vue.js because it helps Vue efficiently identify and track each item in a list. This improves the performance and accuracy of DOM updates.

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app">
      {{suffleEmployeeProjectsOrder()}}
      <p>Projects in Pipeline</p>
      <ul>
        <li
          v-for="(project, index) in employee.projects"
          :key="project.project">
          <span>
            >{{index + 1}} - "{{project.project}}" timeline is -
            {{project.timeline}}</span>
            <input type="text" />
          </li>
        </ul>
      <hr />
    </div>
  </body>
  <script>
    const { createApp, reactive } = Vue;
    createApp({
      setup() {
        const employee = reactive({
          projects: [
            { project: "Green A", timeline: "2 years" },
            { project: "Blue H", timeline: "9 months" },
            { project: "Pink G", timeline: "3 years" },
          ],
        });
        function suffleEmployeeProjectsOrder() {
          setTimeout(() => {
            employee.projects.sort(() => Math.random() - 0.5);
          }, 5000);
        }
        return {
          employee,
          suffleEmployeeProjectsOrder,
        };
      },
    }).mount("#app");
  </script>
</html>
```



Project: Dynamic style bindings

In Vue.js, you can bind styles dynamically to elements using the `v-bind:style` directive. We can easily update styles dynamically based on data properties.

This feature is perfect for creating responsive and interactive designs

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app">
      <div
        v-bind:style="{backgroundColor: 'aqua', fontSize: fontSizeTemp + 'px'}"
      >
        Hello World
      </div>
      <div v-bind:style="{ 'background-color': 'aqua', 'font-size': '40px'}">
        Hello World
      </div>
    </div>
  </body>
  <script>
    const { createApp, ref, reactive } = Vue;
    createApp({
      setup() {
        const fontSizeTemp = 60;
        return {
          fontSizeTemp,
        };
      },
    }).mount("#app");
  </script>
</html>
```

Output:



Project: Class bindings in Vue

In Vue.js, class bindings allow you to dynamically apply CSS classes to elements based on your component's data or computed properties. This is done using the `v-bind:class` directive.

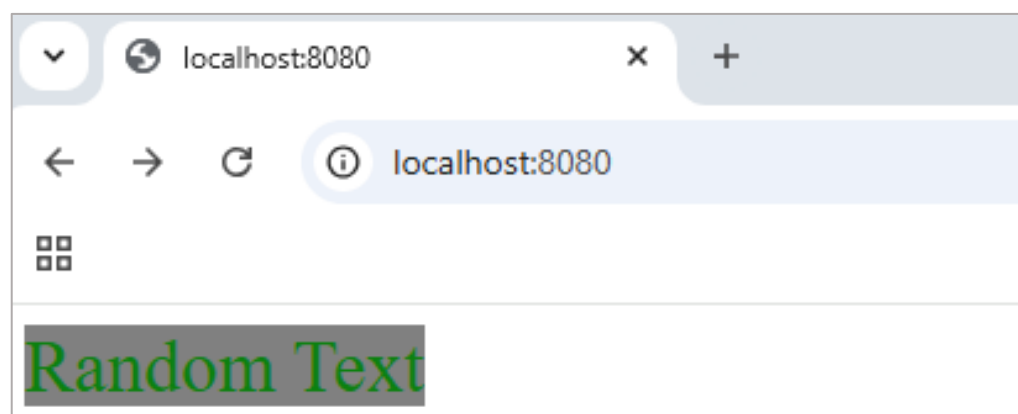
Bind an object where the keys are class names and the values are conditions determining whether the class is applied.

Here `.h1` and `.bg-grey` classes are applied as both conditions are true for both the classes individually.

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app">
      <span class="padding-medium" v-bind:class="{h1: true, 'bg-grey': true}">
        Random Text</span>
    </div>
  </body>
  <script>
    const { createApp, ref, reactive } = Vue;
    createApp({
      setup() {
        return {};
      },
    }).mount("#app");
  </script>
  <style>
    .h1 {
      font-size: 30px;
      color: green;
    }
    .bg-grey {
      background-color: grey;
    }
  </style>
</html>
```

Output:



Project: Array of classes and custom style object

Bind an array of classes, which can include conditional expressions. You can even combine object and array syntax for more flexibility.

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app">
      <span
        class="padding-medium" v-bind:class="[isMainTitle() ? mainTitle : '', borderClass]"
      >Random Text</span>
      <br /><br /><br />
      <span v-bind:style="someStyle.style1">Random Style Binding</span>
    </div>
  </body>
  <script>
    const { createApp } = Vue;
    createApp({
      setup() {
        const mainTitle = {
          h1: true,
          "bg-grey": true,
        };
        const borderClass = {
          border: true,
        };
        const someStyle = {
          style1: {
            fontSize: "40px",
            backgroundColor: "lightgreen",
            margin: "20px",
            border: "1px solid black",
          },
        };
        function isMainTitle() {
          return true;
        }
        return {
          isMainTitle,
          mainTitle,
          someStyle,
          borderClass,
        };
      },
    }).mount("#app");
  </script>
  <style>
    .h1 {
      font-size: 30px;
      color: green;
    }
  </style>
</html>
```

```
}  
.bg-grey {  
  background-color: grey;  
}  
.padding-medium {  
  padding: 20px;  
}  
.border {  
  border: 10px solid red;  
}  
</style>  
</html>
```

Output:



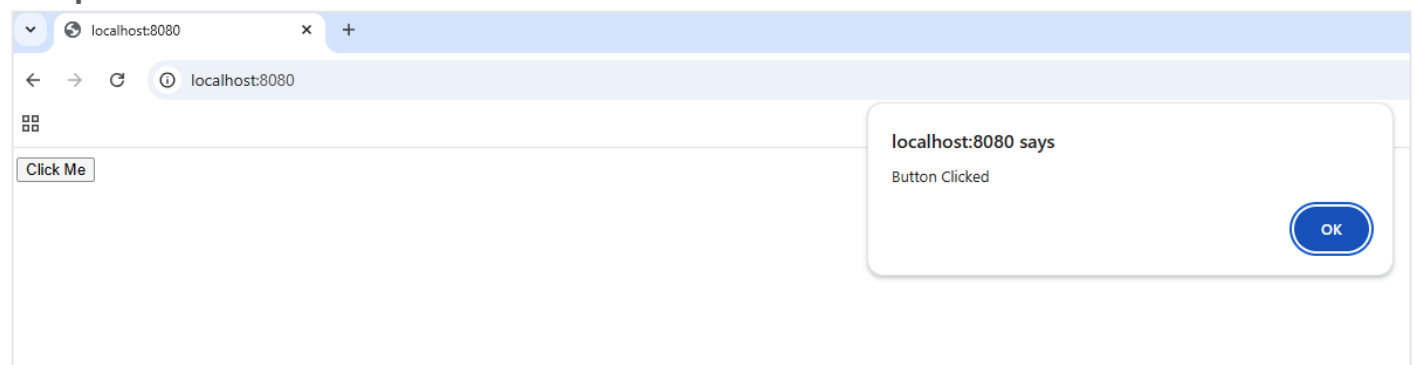
Project: Button on click event

`v-on:click` is a directive in Vue.js used to bind a click event listener to an element. When the element is clicked, the specified method or expression is triggered. It's shorthand for handling click events within Vue's declarative syntax.

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app">
      <button v-on:click="buttonClick">Click Me</button>
    </div>
  </body>
  <script>
    const { createApp } = Vue;
    createApp({
      setup() {
        function buttonClick() {
          alert("Button Clicked");
        }
        return {
          buttonClick,
        };
      },
    }).mount("#app");
  </script>
</html>
```

Output:



Project: Passing parameters to functions

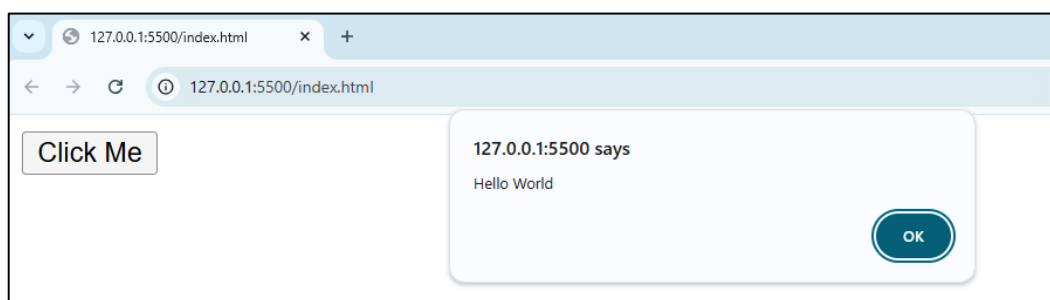
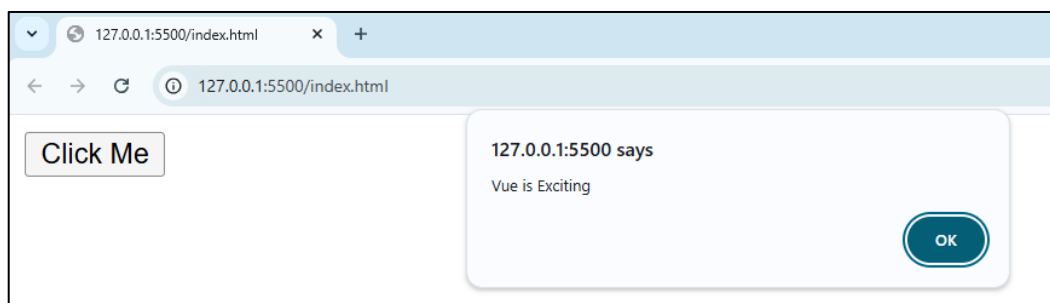
You can easily pass parameters to functions using the directive `v-on:click` in Vue.js

When the button is clicked, the function is called with the parameters “Vue is Exciting” and “Hello World”.

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app">
      <button v-on:click="buttonClick('Vue is Exciting', 'Hello World')">
        Click Me
      </button>
    </div>
  </body>
  <script>
    const { createApp } = Vue;
    createApp({
      setup() {
        function buttonClick(newMessage, newMessage1) {
          alert(newMessage);
          alert(newMessage1);
        }
        return {
          buttonClick,
        };
      },
    }).mount("#app");
  </script>
</html>
```

Output:



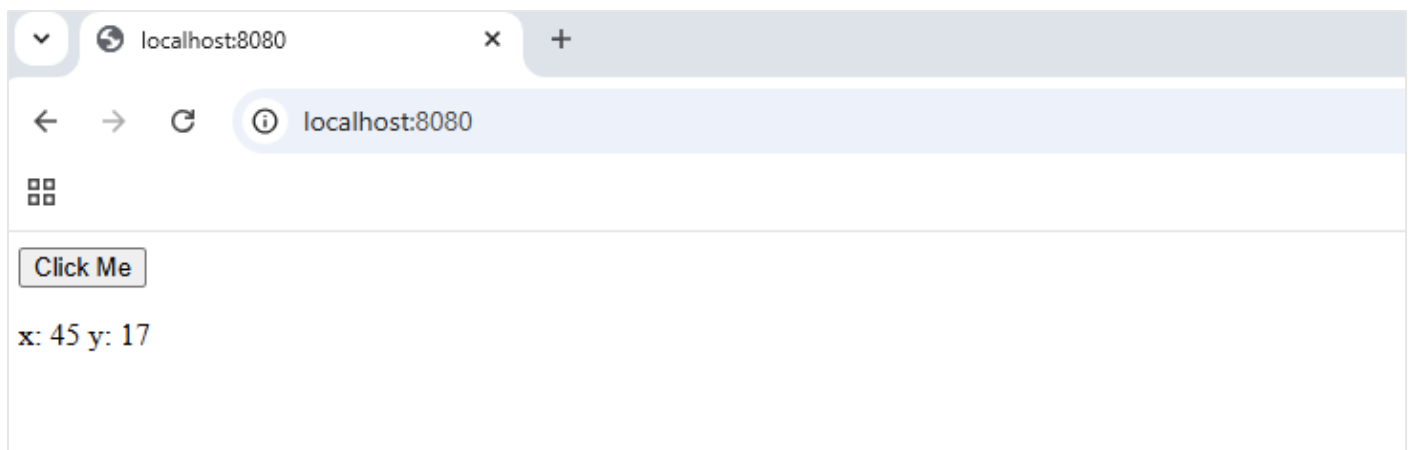
Project: Event Object

In Vue.js, `event` is a special keyword used to access the native DOM event object within event handlers. It's particularly useful when you need to capture details about the event, such as mouse coordinates, key presses, or the event target.

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app">
      <button v-on:click="buttonClick($event)">Click Me</button>
      <p>x: {{x}} y: {{y}}</p>
    </div>
  </body>
  <script>
    const { createApp, ref } = Vue;
    createApp({
      setup() {
        const x = ref(0);
        const y = ref(0);
        function buttonClick(event) {
          x.value = event.x;
          y.value = event.y;
        }
        return {
          x,
          y,
          buttonClick,
        };
      },
    }).mount("#app");
  </script>
</html>
```

Output:



Project: Event modifiers

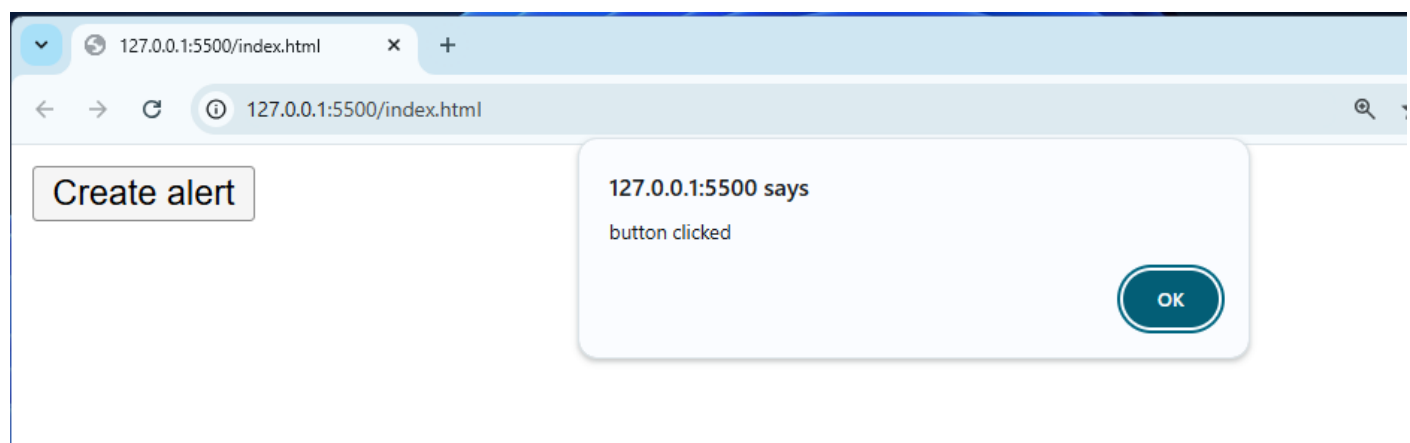
Event modifiers in Vue.js are special keywords that simplify event handling by modifying how events are processed. They can help prevent default behavior, stop event propagation, and much more. You append them to the event handler using a dot.

In the example below if you click the button, it will only work one time. Next time the alert will not be displayed.

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app">
      <button v-on:click.once="buttonClick">Create alert</button>
    </div>
  </body>
  <script>
    const { createApp, ref } = Vue;
    createApp({
      setup() {
        function buttonClick() {
          alert("button clicked");
        }
        return {
          buttonClick,
        };
      },
    }).mount("#app");
  </script>
</html>
```

Output:



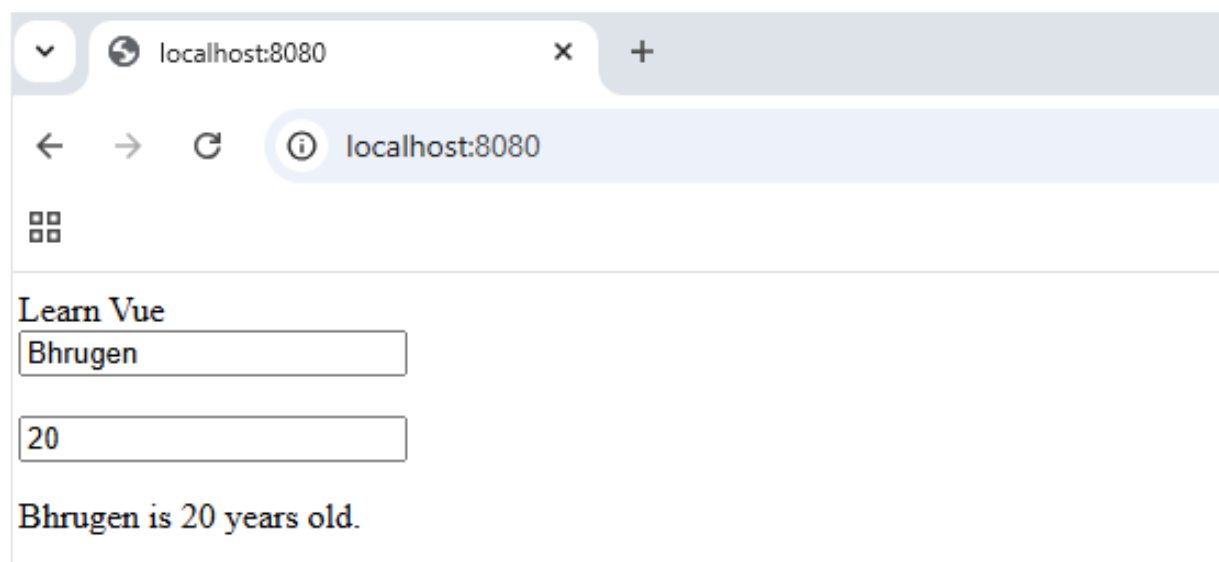
Project: Two-way data binding

Two-way data binding in Vue.js allows for seamless synchronization between the data in your JavaScript model and the DOM. This means any changes in the data update the DOM, and vice versa. It is most commonly implemented using the directive v-model.

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app">
      <div>Learn Vue</div>
      <input type="text" v-model="name" />
      <br />
      <br />
      <input type="number" v-model="age" />
      <p>{{name}} is {{age}} years old.</p>
    </div>
  </body>
  <script>
    const { createApp, ref } = Vue;
    const name = ref("Bhrugen");
    const age = ref(20);
    createApp({
      setup() {
        return {
          name,
          age,
        };
      },
    }).mount("#app");
  </script>
</html>
```

Output:



Project: Model modifiers

Model modifiers in Vue.js are special features that allow you to customize the behaviour of the directive. They provide fine-grained control over how data binding works. These modifiers are appended to with a dot.

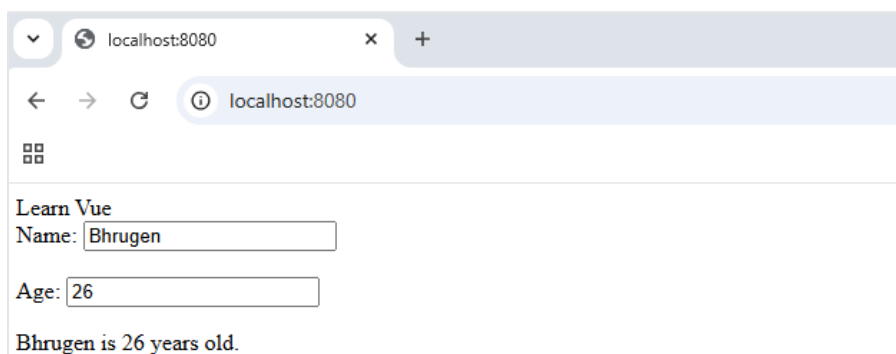
Trim will remove white spaces before and after

Lazy will trigger change only when you click outside and not as soon as you change the value

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app">
      <div>Learn Vue</div>
      Name:
      <input type="text" v-model.trim="name" />
      <br />
      <br />
      Age:
      <input type="number" v-model.lazy="age" />
      <p>{{name}} is {{age}} years old.</p>
    </div>
  </body>
</html>
<script>
  const { createApp, ref } = Vue;
  const name = ref("Bhrugen");
  const age = ref(20);
  createApp({
    setup() {
      return {
        name,
        age,
      };
    },
  }).mount("#app");
</script>
</html>
```

Output:



Project: Computed Properties

In Vue 3, computed properties are a way to define reactive logic that automatically updates based on changes in dependencies.

Computed properties are reactive and only recomputed when their dependencies change.

Computed properties are especially useful for transforming or deriving data efficiently in Vue applications.

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app">
      <p>{{getFullName}}</p>
    </div>
  </body>
  <script>
    const { createApp, ref, computed } = Vue;
    const firstName = ref("Bhrugen");
    const lastName = ref("Patel");
    createApp({
      setup() {
        const getFullName = computed(() => {
          console.log("getFullName was called");
          return firstName.value + " " + lastName.value;
        });
        return {
          getFullName,
        };
      },
    }).mount("#app");
  </script>
</html>
```

Output:



Project: Watchers in Vue

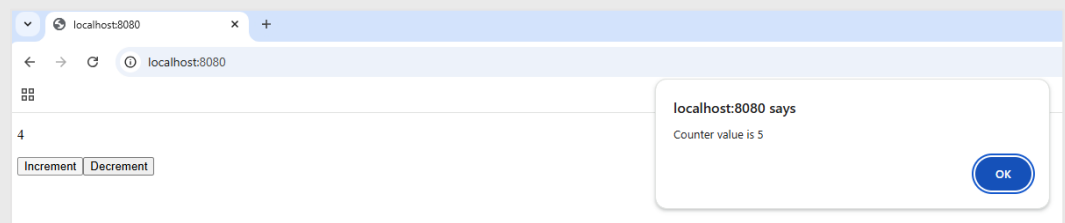
In Vue 3, watchers are used to observe and react to changes in reactive data properties or computed properties. It allows you to run specific logic whenever the observed value changes.

In the example below it will only trigger alert when value of counter reaches 5. It will trigger alert first and then update the value in UI, hence in UI you still see 4.

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app">
      <p>{{counter}}</p>
      <button v-on:click="counter++">Increment</button>
      <button v-on:click="counter--">Decrement</button>
    </div>
  </body>
  <script>
    const { createApp, ref, watch } = Vue;
    let counter = ref(0);
    createApp({
      setup() {
        watch(
          () => counter.value,
          (newValue, oldValue) => {
            console.log(
              "Counter value changed from ",
              oldValue,
              " to ",
              newValue
            );

            if (newValue == 5) {
              alert("Counter value is 5");
            }
          }
        );
        return {
          counter,
        };
      },
    }).mount("#app");
  </script>
</html>
```



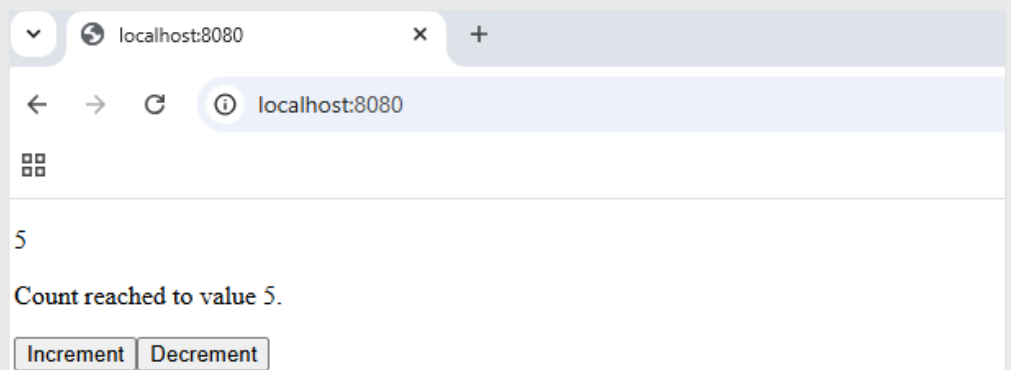
Project: Watchers can be asynchronous

In Vue 3, you can use asynchronous watchers to handle logic that involves asynchronous operations, such as API calls, delays, or other promises.

This approach makes it easy to integrate asynchronous functionality into your reactive logic in Vue 3. In the example below once counter reaches 5 the message will be displayed after waiting for 3 sec.

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app">
      <p>{{obj.count}}</p>
      <p v-if="showMessage">Count reached to value 5.</p>
      <button v-on:click="obj.count++">Increment</button>
      <button v-on:click="obj.count--">Decrement</button>
    </div>
  </body>
  <script>
    const { createApp, ref, reactive, watch } = Vue;
    const obj = reactive({ count: 0 });
    const showMessage = ref(false);
    createApp({
      setup() {
        watch(
          () => obj.count,
          (newValue, oldValue) => {
            if (newValue == 5) {
              setTimeout(() => {
                showMessage.value = true;
              }, 3000);
            }
          }
        );
        return {
          showMessage,
          obj,
        };
      },
    }).mount("#app");
  </script>
</html>
```



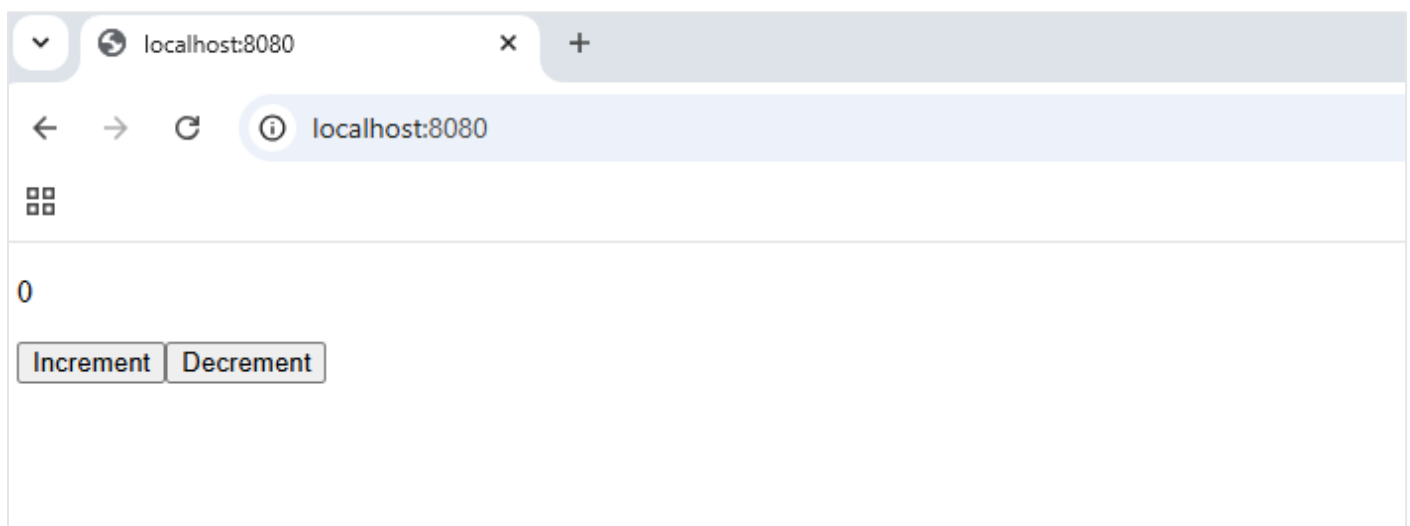
Project: Shortcut for v-on

In Vue 3, shorthand for events simplifies the syntax for event handling. Instead of using the full directive , you can use the symbol @ as a shorthand for v-on.

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app">
      <p>{{obj.count}}</p>
      <button @click="obj.count++">Increment</button>
      <button @click="obj.count--">Decrement</button>
    </div>
  </body>
  <script>
    const { createApp, reactive } = Vue;
    const obj = reactive({ count: 0 });
    createApp({
      setup() {
        return {
          obj,
        };
      },
    }).mount("#app");
  </script>
</html>
```

Output:

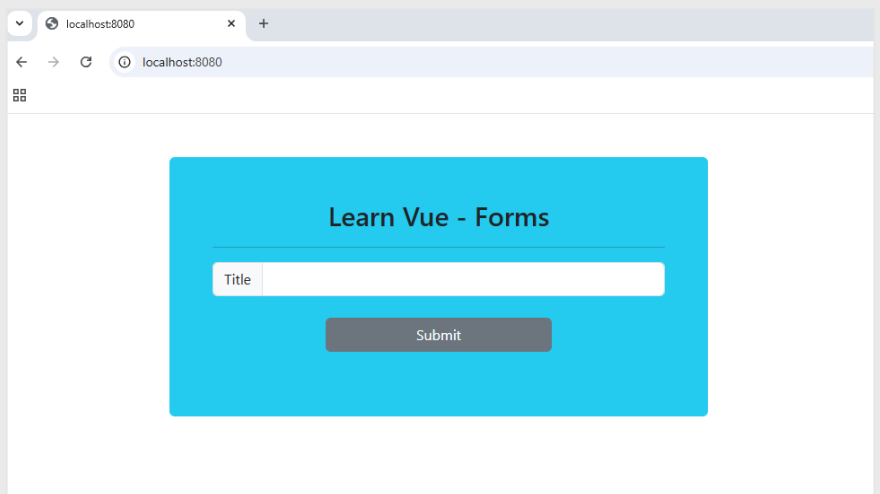


Project: Forms setup

In Vue 3, forms are primarily managed using the directive, which enables two-way data binding for form inputs. Vue 3 enhances form handling with modifiers and reactive properties.

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <link
      href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
      rel="stylesheet"
      integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh0JMhY6hW+ALEwIH"
      crossorigin="anonymous"
    />
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app" class="container">
      <form>
        <div class="bg-info p-5 rounded m-5">
          <div class="h2 text-center">Learn Vue - Forms</div>
          <hr />
          <div class="input-group mb-3">
            <span class="input-group-text">Title</span>
            <input type="text" class="form-control" />
          </div>
          <div class="text-center pb-3">
            <button class="btn btn-secondary m-2 w-50">Submit</button>
          </div>
        </div>
      </form>
    </div>
  </body>
  <script>
    const { createApp, reactive } = Vue;
    const formObj = reactive({
      email: "",
      name: "",
      phone: "",
      subscribe: false,
      gender: "",
    });
    createApp({
      setup() {
        return {
          formObj,
        };
      },
    }).mount("#app");
  </script>
</html>
```



Project: Prevent submit in forms

In Vue 3, the event modifier is used to prevent the default browser behaviour for events, such as form submissions or link clicks. This is especially useful for handling form submissions using JavaScript rather than reloading the page.

You can also use model modifiers (trim/number) along with v-model directive.

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <link
      href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
      crossorigin="anonymous"
      rel="stylesheet"
      integrity="sha384-
QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh0JMhY6hW+ALEwIH" />
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app" class="container">
      <form v-on:submit.prevent="handleSubmit">
        <div class="bg-info p-5 rounded m-5">
          <div class="h2 text-center">Learn Vue - Forms</div>
          <hr />
          <div class="input-group mb-3">
            <span class="input-group-text">Email</span>
            <input type="text" class="form-control" v-model.trim="formObj.email" />
          </div>
          <div class="input-group mb-3">
            <span class="input-group-text">Name</span>
            <input
              type="text" class="form-control" v-model.trim="formObj.name" />
          </div>
          <div class="input-group mb-3">
            <span class="input-group-text">Phone</span>
            <input type="number" class="form-control" v-model.number="formObj.phone" />
          </div>
          <div class="border p-1">
            <label class="form-label">Gender</label><br />
            <div class="form-check form-check-inline">
              <input class="form-check-input" type="radio"
                v-model="formObj.gender" value="male" name="inlineRadioOptions">Male</input>
            </div>
            <div class="form-check form-check-inline">
              <input class="form-check-input" type="radio"
                v-model="formObj.gender" value="female" name="inlineRadioOptions">Female</input>
            </div>
            <br />
            <select id="gender" v-model="formObj.gender" class="form-select">
              <option value="male">Male</option>
              <option value="female">Female</option>
            </select>
          </div>
          <div class="input-group mb-3">
            <input
              type="checkbox" class="form-check-input" v-model="formObj.subscribe" />
            <label class="form-check-label">Subscribe to newsletter</label>
          </div>
        </div>
      </form>
    </div>
  </body>
</html>
```



```

    >
  </div>
  <div class="text-center pb-3">
    <button class="btn btn-secondary m-2 w-50">Submit</button>
  </div>
</div>
</form>
</div>
</body>
<script>
const { createApp, reactive } = Vue;
const formObj = reactive({
  email: "",
  name: "",
  phone: "",
  subscribe: false,
  gender: "male",
});
function handleSubmit() {
  console.log(formObj);
}
createApp({
  setup() {
    return {
      handleSubmit,
      formObj,
    };
  },
}).mount("#app");
</script>
</html>

```

Output:

The screenshot shows a web browser window with the address bar displaying 'localhost:8080'. The page content is a form titled 'Learn Vue - Forms' on a light blue background. The form includes the following elements:

- Input field for 'Email'.
- Input field for 'Name'.
- Input field for 'Phone'.
- Gender selection section with radio buttons for 'Male' (selected) and 'Female'.
- A dropdown menu for gender selection, currently displaying 'Male'.
- An unchecked checkbox labeled 'Subscribe to newsletter'.
- A dark grey 'Submit' button at the bottom center.

Project: Forms Validation

In Vue 3, form validation can be efficiently handled using custom logic, directives etc.

Source Code:

```
<!DOCTYPE html>
<html>
  <head>
    <link
      href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" crossorigin="anonymous"
      rel="stylesheet" integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh0JMhY6hW+ALEwIH" />
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  </head>
  <body>
    <div id="app" class="container">
      <form v-on:submit.prevent="handleSubmit">
        <div class="bg-warning p-5 rounded m-5">
          <div class="h2 text-center">Learn Vue - FormsOPedia</div>
          <hr />
          <div>
            <ul>
              <li v-for="error in errorList" class="text-danger">{{error}}</li>
            </ul>
          </div>
          <div class="input-group mb-3">
            <span class="input-group-text">Email</span>
            <input type="text" class="form-control" v-model.trim="formObj.email" />
          </div>
          <div class="input-group mb-3">
            <span class="input-group-text">Name</span>
            <input type="text" class="form-control" v-model.trim="formObj.name" />
          </div>

          <div class="input-group mb-3">
            <input type="checkbox" class="form-check-input" v-model="formObj.subscribe" />
            <label class="form-check-label" > Subscribe to newsletter</label>
          </div>
          <div class="text-center pb-3">
            <button class="btn btn-secondary m-2 w-50">Submit</button>
          </div>
        </div>
      </form>
    </div>
  </body>
  <script>
    const { createApp, reactive } = Vue;
    const errorList = reactive([]);
    const formObj = reactive({
      email: "",
      name: "",
      subscribe: false,
    });
    function handleSubmit() {
      errorList.length = 0;
      if (formObj.name.length < 3) {
        errorList.push("Name should be at least 3 character long.");
      }
    }
  </script>
</script>
```

```

    }
    if (formObj.email === "") {
      errorList.push("Email is required.");
    }
  }
  if (!formObj.subscribe) {
    errorList.push("Please subscribe to newsletter.");
  }
  if (!errorList.length) {
    alert("Form submitted successfully.");
  }
  console.log(formObj);
}
createApp({
  setup() {
    return {
      errorList,
      handleSubmit,
      formObj,
    };
  },
}).mount("#app");
</script>
</html>

```

Output:



Learn Vue - FormsOPedia

- Name should be at least 3 character long.
- Email is required.
- Please subscribe to newsletter.

Email

Name

☐ Subscribe to newsletter

Submit

Project: Create first component

Project is created using Vite. To create or to define a component, create a .vue file and use the <template>, <script> and <style> tags to define your component.

Now register the component locally in the component where you want use it or globally in main.js file.

Source Code:

src/main.js:

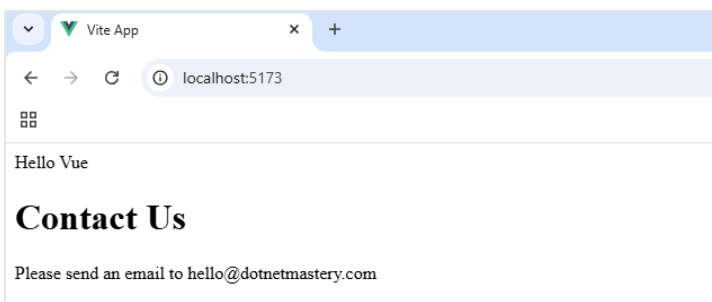
```
import { createApp } from 'vue'
import App from './App.vue'
import ContactUs from './components/ContactUs.vue'
const app = createApp(App)
app.component('contact-us', ContactUs)
app.mount('#app')
```

src/App.vue:

```
<template>
  <div>{{ message }}</div>
  <contact-us></contact-us>
</template>
<script>
export default {
  setup() {
    const message = 'Hello Vue'
    return {
      message,
    }
  },
}
</script>
<style></style>
```

src/components/ContactUs.vue:

```
<template>
  <div>
    <h1>Contact Us</h1>
    <p>Please send an email to hello@dotnetmastery.com</p>
  </div>
</template>
```



Project: Cleaner setup

<script setup> is a syntax designed for simpler and faster composition of Vue components using the Composition API. It eliminates boilerplate code like export default.

We define reactive state, functions, and lifecycle hooks directly within the <script setup> tag.

Source Code:

src/main.js:

```
import { createApp } from 'vue'
import App from './App.vue'
import ContactUs from './components/ContactUs.vue'
import ButtonCounter from './components/ButtonCounter.vue'
const app = createApp(App)
app.component('contact-us', ContactUs)
app.component('button-counter', ButtonCounter)
app.mount('#app')
```

src/App.vue:

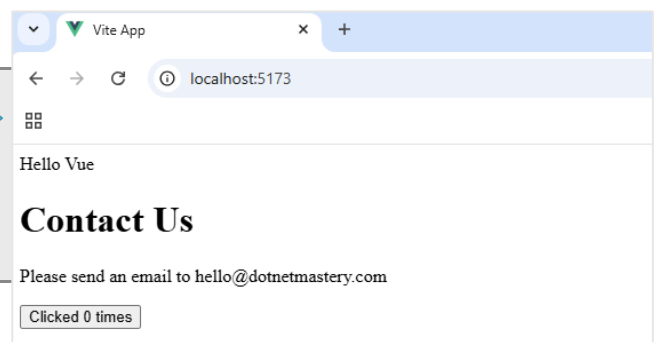
```
<template>
  <div>{{ message }}</div>
  <contact-us></contact-us>
  <button-counter></button-counter>
</template>
<script setup>
const message = 'Hello Vue'
</script>
```

src/components/ContactUs.vue

```
<template>
  <div>
    <h1>Contact Us</h1>
    <p>Please send an email to {{ email }}</p>
  </div>
</template>
<script setup>
import { ref } from 'vue'
const email = ref('hello@dotnetmastery.com')
</script>
```

src/components/ButtonCounter.vue

```
<template>
  <button @click="count++">Clicked {{ count }} times</button>
</template>
<script setup>
import { ref } from 'vue'
const count = ref(0)
</script>
```



Project: Local vs Global Components

Local components are registered within a specific component and it is accessible only in the component where it's registered. Local components are used in a limited scope or specific part of your application to avoid polluting the global namespace.

Global components are registered globally across the entire app. Global components are accessible in all components without explicitly importing or registering. Global components are reusable across multiple parts of the app, like buttons, modals, or loaders.

Source Code:

src/main.js:

```
import { createApp } from 'vue'
import App from './App.vue'
import ContactUs from './components/ContactUs.vue'
const app = createApp(App)
// registering global components
app.component('contact-us', ContactUs)
app.mount('#app')
```

src/App.vue

```
<template>
  <div>{{ message }}</div>
  <contact-us></contact-us>
  <!-- local component -->
  <ButtonCounter></ButtonCounter>
</template>
<script setup>
import ButtonCounter from './components/ButtonCounter.vue'
const message = 'Hello Vue'
</script>
<style></style>
```

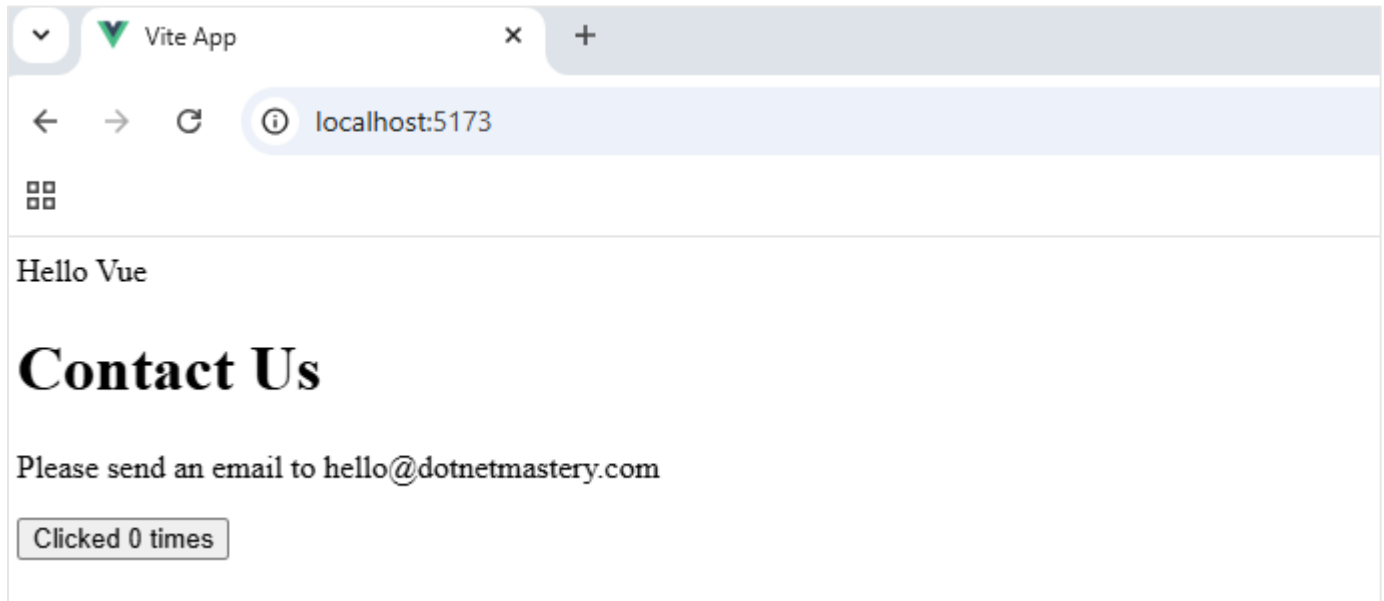
src/components/ContactUs.vue:

```
<template>
  <div>
    <h1>Contact Us</h1>
    <p>Please send an email to {{ email }}</p>
  </div>
</template>
<script setup>
import { ref } from 'vue'
const email = ref('hello@dotnetmastery.com')
</script>
<style></style>
```

src/components/ButtonCounter.vue

```
<template>
  <button @click="count++">Clicked {{ count }} times</button>
</template>
<script setup>
import { ref } from 'vue'
const count = ref(0)
</script>
<style></style>
```

Output:



Project: Passing Props

In Vue 3, props are a way to pass data from a parent component to a child component.

We can define Props in the Child Component using the defineProps function in <script setup>.

Source Code:

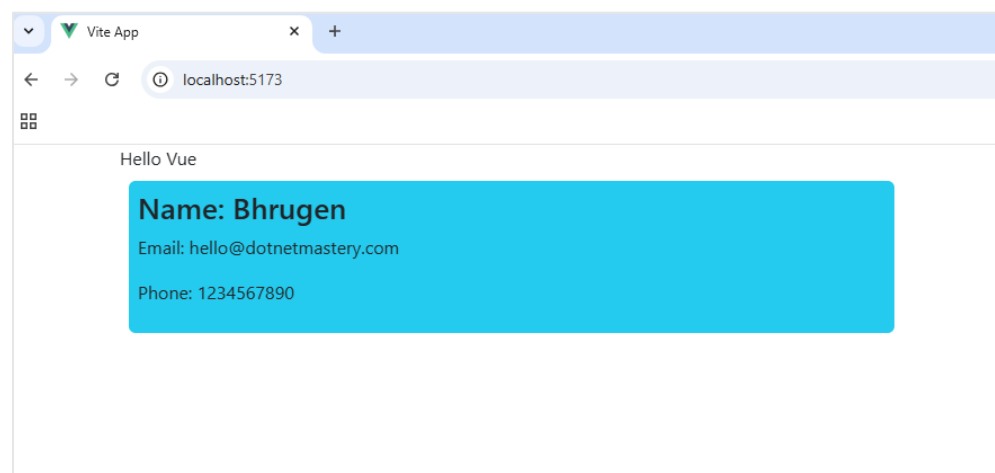
src/App.vue:

```
<template>
  <div class="container">
    <div>{{ message }}</div>
    <Contact name="Bhrugen" phone="1234567890"></Contact>
  </div>
</template>
<script setup>
import Contact from './components/Contact.vue'
const message = 'Hello Vue'
</script>
<style></style>
```

src/components/Contact.vue:

```
<template>
  <div class="bg-info rounded p-2 m-2">
    <h3>Name: {{ name }}</h3>
    <p>Email: {{ email }}</p>
    <p>Phone: {{ phone }}</p>
  </div>
</template>
<script setup>
import { ref } from 'vue'
const props = defineProps(['name', 'phone'])
const email = ref('hello@dotnetmastery.com')
</script>
<style></style>
```

Output:



Project: Binded props

Props allow you to pass data from a parent component to a child component. Vue 3 simplifies binding props through the `v-bind` directive.

Source Code:

Src/main.js:

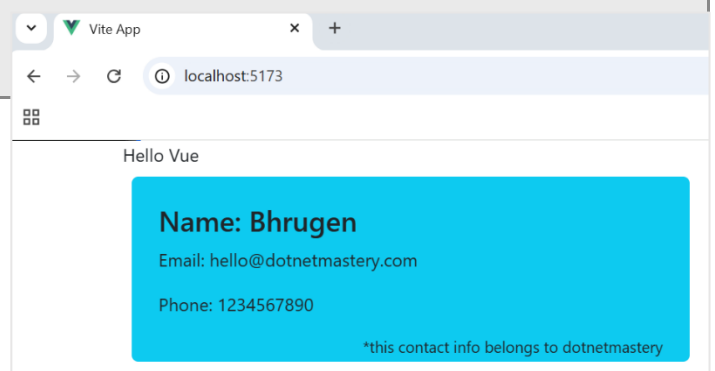
```
import { createApp } from 'vue'
import App from './App.vue'
const app = createApp(App)
app.mount('#app')
```

src/App.vue:

```
<template>
  <div class="container">
    <div>{{ message }}</div>
    <Contact name="Bhrugen" phone="1234567890" :ownername="contactsOwner"></Contact>
  </div>
</template>
<script setup>
import Contact from './components/Contact.vue'
const message = 'Hello Vue'
const contactsOwner = 'dotnetmastery'
</script>
<style></style>
```

Src/components/Contact.vue:

```
<template>
  <div class="bg-info rounded p-4 m-2">
    <h3>Name: {{ name }}</h3>
    <p>Email: {{ email }}</p>
    <p>Phone: {{ phone }}</p>
    <p class="float-end small" v-if="ownername !== ''">
      *this contact info belongs to {{ ownername }}
    </p>
  </div>
</template>
<script setup>
import { ref } from 'vue'
const props = defineProps(['name', 'phone', 'ownername'])
const email = ref('hello@dotnetmastery.com')
</script>
<style></style>
```



Project: Validating props

In Vue 3, props validation ensures that the props passed to a component are of the correct type, format, or value.

Source Code:

Src/main.js:

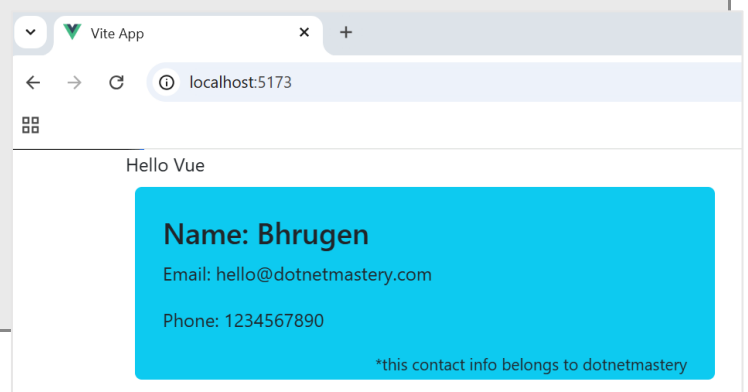
```
import { createApp } from 'vue'
import App from './App.vue'
const app = createApp(App)
app.mount('#app')
```

src/App.vue:

```
<template>
  <div class="container">
    <div>{{ message }}</div>
    <Contact name="Bhrugen" :phone="1234567890" :ownername="contactsOwner"></Contact>
  </div>
</template>
<script setup>
import Contact from './components/Contact.vue'
const message = 'Hello Vue'
const contactsOwner = 'dotnetmastery'
</script>
<style></style>
```

Src/components/Contact.vue:

```
<template>
  <div class="bg-info rounded p-4 m-2">
    <h3>Name: {{ name }}</h3>
    <p>Email: {{ email }}</p>
    <p>Phone: {{ phone }}</p>
    <p class="float-end small" v-if="ownername !== ''">
      *this contact info belongs to {{ ownername }}
    </p>
  </div>
</template>
<script setup>
import { ref } from 'vue'
const props = defineProps({
  name: String,
  phone: Number,
  ownername: String,
})
const email = ref('hello@dotnetmastery.com')
</script>
<style></style>
```



Project: props and local variable

Using the same name for props and local variables in Vue 3 is possible but can lead to confusion. If you have a prop and a local variable with the same name, the local variable takes precedence in templates. This means the prop is overshadowed, and you'll lose direct access to it.

Avoid using the same name for props and local variables to maintain clarity and readability. Use unique, descriptive names for local variables and props to prevent overshadowing.

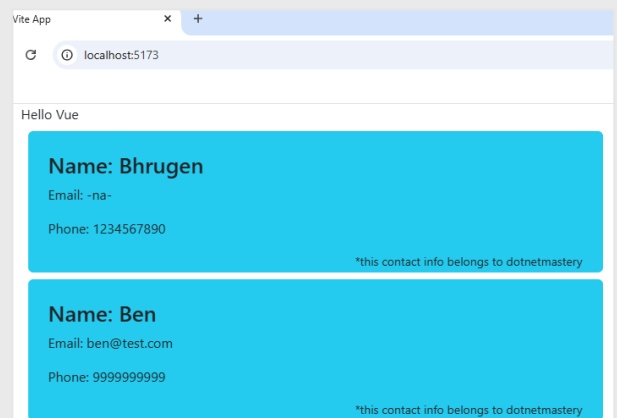
Source Code:

Src/App.vue:

```
<template>
  <div class="container">
    <div>{{ message }}</div>
    <Contact name="Bhrugen" :phone="1234567890" :ownername="contactsOwner"></Contact>
    <Contact
      name="Ben"
      :phone="9999999999"
      :ownername="contactsOwner"
      email="ben@test.com"
    ></Contact>
  </div>
</template>
<script setup>
import Contact from './components/Contact.vue'
const message = 'Hello Vue'
const contactsOwner = 'dotnetmastery'
</script>
```

Src/components/Contact.vue:

```
<template>
  <div class="bg-info rounded p-4 m-2">
    <h3>Name: {{ name }}</h3>
    <p>Email: {{ email }}</p>
    <p>Phone: {{ phone }}</p>
    <p class="float-end small" v-if="ownername !== ''">
      *this contact info belongs to {{ ownername }}
    </p>
  </div>
</template>
<script setup>
const props = defineProps({
  name: {
    type: String, required: true,
  },
  phone: {
    type: Number, required: true,
  },
  ownername: String,
  email: {
    type: String, required: false, default: '-na-',
  },
})
</script>
```



Project: Passing dynamic prop binding

Source Code:

Src/App.vue:

```
<template>
  <div class="container">
    Contact Owner Name: <input type="text" v-model="ownerName" />
    <div class="row">
      <div class="col-12" v-for="contact in contacts" :key="contact.name">
        <Contact
          :name="contact.name"
          :phone="contact.phone"
          :ownername="contact.ownerName"
          :email="contact.email"
          :isFavorite="contact.isFavorite"
        ></Contact>
      </div>
    </div>
  </div>
</template>
<script setup>
import { ref, reactive } from 'vue'
import Contact from './components/Contact.vue'
const ownerName = ref('dotnetmastery')
const contacts = reactive([
  { name: 'Bhrugen', phone: 1234567890, ownerName: ownerName, isFavorite: false },
  { name: 'Bella', phone: 5555555555, ownerName: ownerName, isFavorite: true },
  {
    name: 'Ben',
    phone: 2222222222,
    ownerName: ownerName,
    email: 'ben@dotnetmastery.com',
    isFavorite: false,
  },
])
</script>
<style></style>
```

Src/components/Contact.vue:

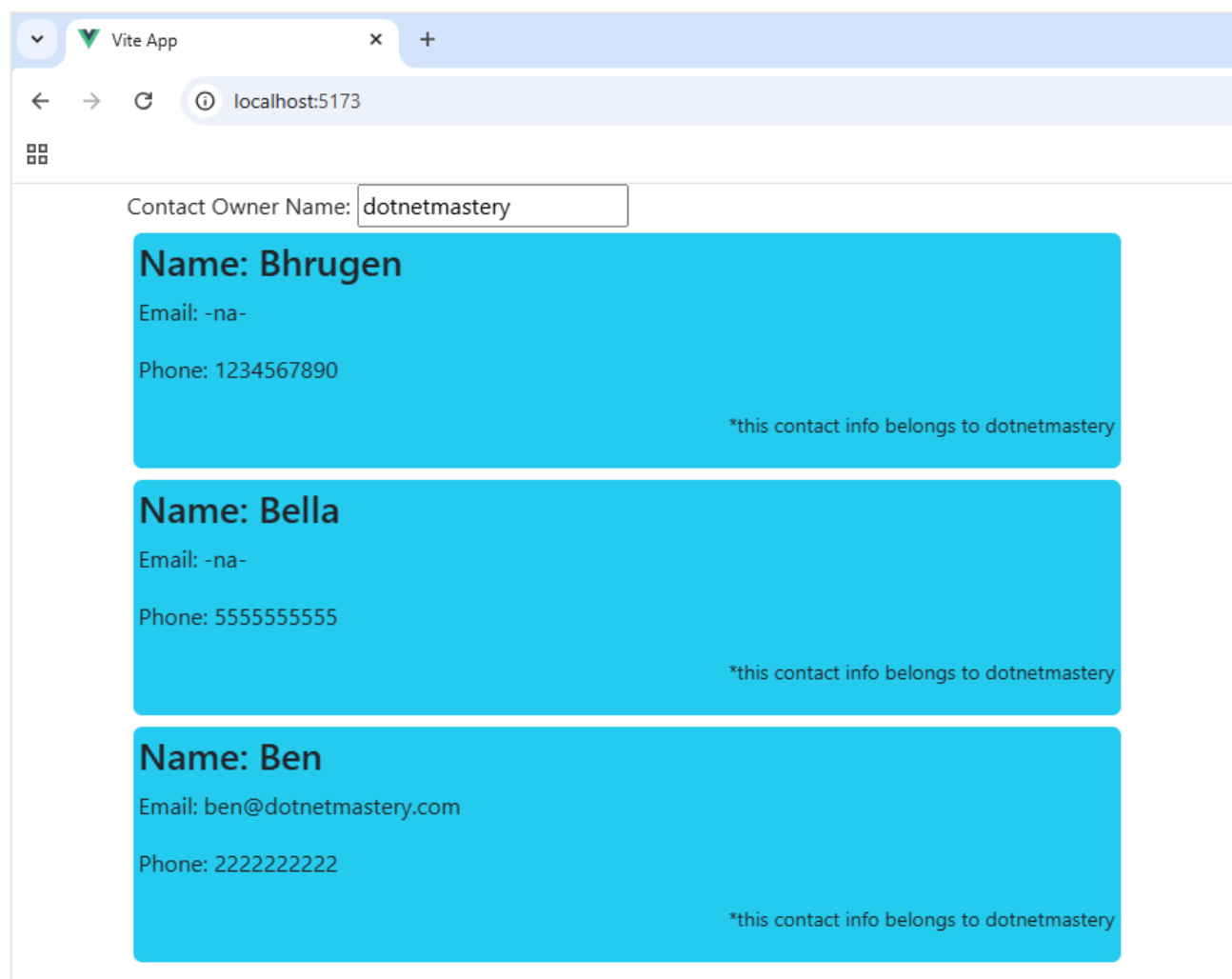
```
<template>
  <div class="bg-info rounded p-1 m-1">
    <div class="row">
      <div class="col-12">
        <h3>Name: {{ name }}</h3>
        <p>Email: {{ email }}</p>
        <p>Phone: {{ phone }}</p>
        <p class="float-end small" v-if="ownername !== ''">
          *this contact info belongs to {{ ownername }}
        </p>
      </div>
    </div>
  </div>
</template>
```

```

</div>
</template>
<script setup>
const props = defineProps({
  name: {
    type: String,
    required: true,
  },
  phone: {
    type: Number,
    required: true,
  },
  ownername: String,
  email: {
    type: String,
    required: false,
    default: '-na-',
  },
  isFavorite: Boolean,
})
</script>
<style></style>

```

Output:



Project: Dynamically change class and button text

Source Code:

Src/App.vue:

```
<template>
  <div class="container">
    Contact Owner Name: <input type="text" v-model="ownerName" />
    <div class="row">
      <div class="col-12" v-for="contact in contacts" :key="contact.name">
        <Contact
          :name="contact.name"
          :phone="contact.phone"
          :ownername="contact.ownerName"
          :email="contact.email"
          :isFavorite="contact.isFavorite"
        ></Contact>
      </div>
    </div>
  </div>
</template>
<script setup>
import { ref, reactive } from 'vue'
import Contact from './components/Contact.vue'
const ownerName = ref('dotnetmastery')
const contacts = reactive([
  { name: 'Bhrugen', phone: 1234567890, ownerName: ownerName, isFavorite: false },
  { name: 'Bella', phone: 5555555555, ownerName: ownerName, isFavorite: true },
  {
    name: 'Ben',
    phone: 2222222222,
    ownerName: ownerName,
    email: 'ben@dotnetmastery.com',
    isFavorite: false,
  },
])
</script>
<style></style>
```

Src/components/Contact.vue:

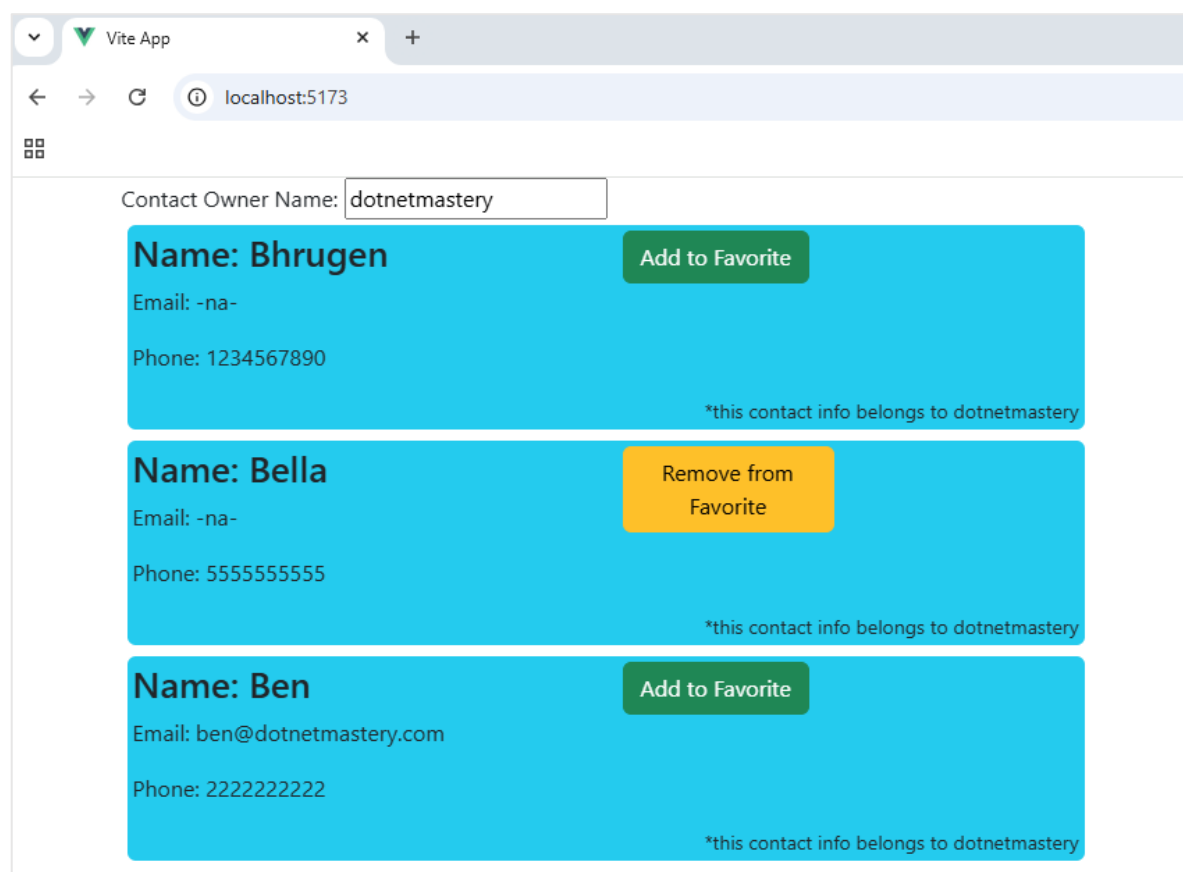
```
<template>
  <div class="bg-info rounded p-1 pb-4 m-1">
    <div class="row">
      <div class="col-6">
        <h3>Name: {{ name }}</h3>
        <p>Email: {{ email }}</p>
        <p>Phone: {{ phone }}</p>
      </div>
      <div class="col-3">
        <button :class="[isFavorite ? 'btn btn-warning' : 'btn btn-success']">
          {{ isFavorite ? 'Remove from' : 'Add to' }} Favorite
        </button>
      </div>
    </div>
  </div>
</template>
```

```

    </div>
  </div>
  <span class="float-end small" v-if="ownername !== ''">
    *this contact info belongs to {{ ownername }}
  </span>
</div>
</template>
<script setup>
const props = defineProps({
  name: {
    type: String,
    required: true,
  },
  phone: {
    type: Number,
    required: true,
  },
  ownername: String,
  email: {
    type: String,
    required: false,
    default: '-na-',
  },
  isFavorite: Boolean,
})
</script>
<style></style>

```

Output:



Project: Emits in Action

Emit is used in a child component to send custom events to a parent component, allowing communication from child to parent.

Source Code:

Src/App.vue:

```
<template>
  <div class="container">
    Contact Owner Name: <input type="text" v-model="ownerName" />
    <div class="row">
      <div class="col-12" v-for="contact in contacts" :key="contact.name">
        <Contact
          :name="contact.name"
          :phone="contact.phone"
          :ownername="contact.ownerName"
          :email="contact.email"
          :isFavorite="contact.isFavorite"
          @update-favorite="contact.isFavorite = !contact.isFavorite"
        ></Contact>
      </div>
    </div>
  </div>
</template>
<script setup>
import { ref, reactive } from 'vue'
import Contact from './components/Contact.vue'
const ownerName = ref('dotnetmastery')
const contacts = reactive([
  { name: 'Bhrugen', phone: 1234567890, ownerName: ownerName, isFavorite: false },
  { name: 'Bella', phone: 5555555555, ownerName: ownerName, isFavorite: true },
  {
    name: 'Ben',
    phone: 2222222222,
    ownerName: ownerName,
    email: 'ben@dotnetmastery.com',
    isFavorite: false,
  },
])
</script>
<style></style>
```

Src/components/Contact.vue:

```
<template>
  <div class="bg-info rounded p-1 pb-4 m-1">
    <div class="row">
      <div class="col-6">
        <h3>Name: {{ name }}</h3>
        <p>Email: {{ email }}</p>
        <p>Phone: {{ phone }}</p>
      </div>
      <div class="col-3">
        <button
          @click="toggleFavorite"
        >
```



```

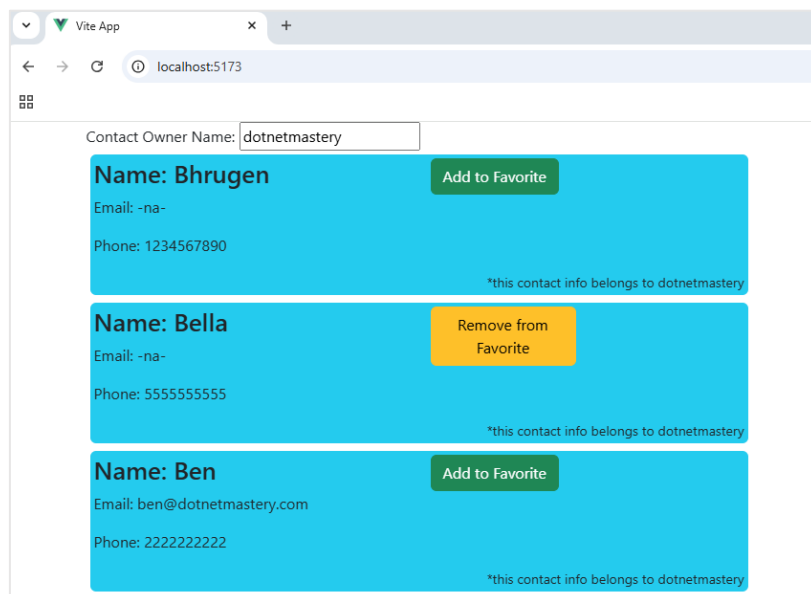
      :class="[isFavorite ? 'btn btn-warning' : 'btn btn-success']"
    >
      {{ isFavorite ? 'Remove from' : 'Add to' }} Favorite
    </button>
  </div>
</div>
<span class="float-end small" v-if="ownername !== ''">
  *this contact info belongs to {{ ownername }}
</span>
</div>
</template>
<script setup>
const props = defineProps({
  name: {
    type: String,
    required: true,
  },
  phone: {
    type: Number,
    required: true,
  },
  ownername: String,
  email: {
    type: String,
    required: false,
    default: '-na-',
  },
  isFavorite: Boolean,
})

const emit = defineEmits(['update-favorite'])

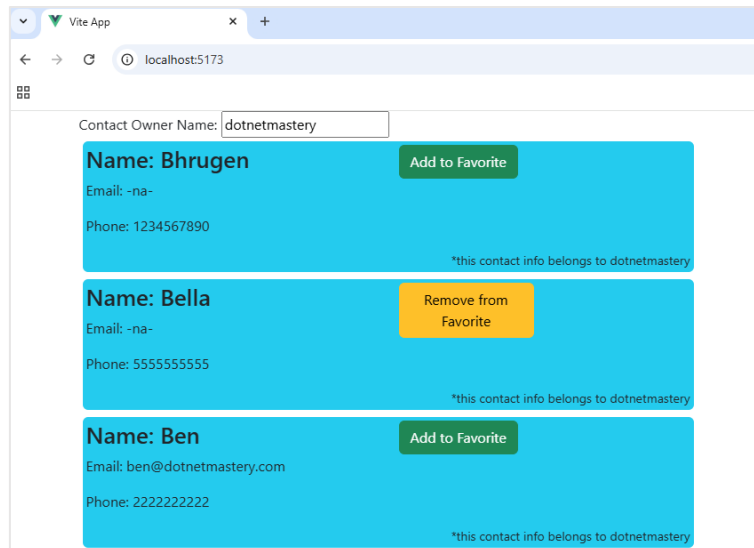
function toggleFavorite() {
  emit('update-favorite')
}
</script>
<style></style>

```

Output:



Project: Passing parameters in emits



Src/App.vue:

```
<template>
  <div class="container">
    Contact Owner Name: <input type="text" v-model="ownerName" />
    <div class="row">
      <div class="col-12" v-for="contact in contacts" :key="contact.name">
        <Contact
          :name="contact.name"
          :phone="contact.phone"
          :ownername="contact.ownerName"
          :email="contact.email"
          :isFavorite="contact.isFavorite"
          @update-favorite="contact.isFavorite = onUpdateFavorite($event)"
        ></Contact>
      </div>
    </div>
  </div>
</template>
<script setup>
import { ref, reactive } from 'vue'
import Contact from './components/Contact.vue'
const ownerName = ref('dotnetmastery')
const contacts = reactive([
  { name: 'Bhrugen', phone: 1234567890, ownerName: ownerName, isFavorite: false },
  { name: 'Bella', phone: 5555555555, ownerName: ownerName, isFavorite: true },
  {
    name: 'Ben',
    phone: 2222222222,
    ownerName: ownerName,
    email: 'ben@dotnetmastery.com',
    isFavorite: false,
  },
])

function onUpdateFavorite(isFavoriteFromChildComponent) {
  return !isFavoriteFromChildComponent
}
</script>
```

```

<template>
  <div class="bg-info rounded p-1 pb-4 m-1">
    <div class="row">
      <div class="col-6">
        <h3>Name: {{ name }}</h3>
        <p>Email: {{ email }}</p>
        <p>Phone: {{ phone }}</p>
      </div>
      <div class="col-3">
        <button
          @click="toggleFavorite"
          :class="[isFavorite ? 'btn btn-warning' : 'btn btn-success']"
        >
          {{ isFavorite ? 'Remove from' : 'Add to' }} Favorite
        </button>
      </div>
    </div>
    <span class="float-end small" v-if="ownername !== ''">
      *this contact info belongs to {{ ownername }}
    </span>
  </div>
</template>
<script setup>
const props = defineProps({
  name: {
    type: String,
    required: true,
  },
  phone: {
    type: Number,
    required: true,
  },
  ownername: String,
  email: {
    type: String,
    required: false,
    default: '-na-',
  },
  isFavorite: Boolean,
})

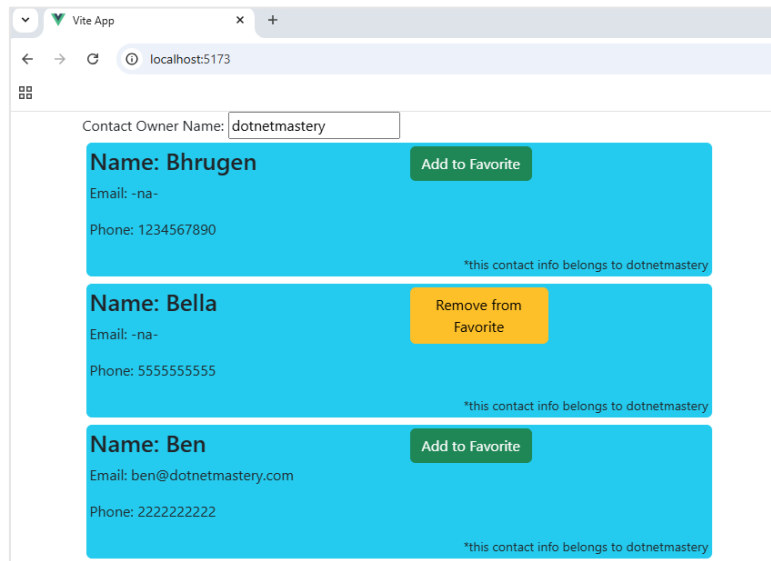
const emit = defineEmits(['update-favorite'])

function toggleFavorite() {
  emit('update-favorite', props.isFavorite)
}
</script>

```

Project: Passing objects in Emit

To pass objects with emit in Vue 3, you simply include the object as a parameter in the emit function.



Src/App.vue:

```
<template>
  <div class="container">
    Contact Owner Name: <input type="text" v-model="ownerName" />
    <div class="row">
      <div class="col-12" v-for="contact in contacts" :key="contact.name">
        <Contact
          :name="contact.name"
          :phone="contact.phone"
          :ownername="contact.ownerName"
          :email="contact.email"
          :isFavorite="contact.isFavorite"
          @update-favorite="contact.isFavorite = onUpdateFavorite($event)"
        ></Contact>
      </div>
    </div>
  </div>
</template>
<script setup>
import { ref, reactive } from 'vue'
import Contact from './components/Contact.vue'
const ownerName = ref('dotnetmastery')
const contacts = reactive([
  { name: 'Bhruugen', phone: 1234567890, ownerName: ownerName, isFavorite: false },
  { name: 'Bella', phone: 5555555555, ownerName: ownerName, isFavorite: true },
  {
    name: 'Ben', phone: 2222222222, ownerName: ownerName,
    email: 'ben@dotnetmastery.com', isFavorite: false,
  },
],
])

function onUpdateFavorite(oldValuesFromChildComponent) {
  console.log(oldValuesFromChildComponent.name)
  return !oldValuesFromChildComponent.isFavorite
}
</script>
```

```

<template>
  <div class="bg-info rounded p-1 pb-4 m-1">
    <div class="row">
      <div class="col-6">
        <h3>Name: {{ name }}</h3>
        <p>Email: {{ email }}</p>
        <p>Phone: {{ phone }}</p>
      </div>
      <div class="col-3">
        <button
          @click="toggleFavorite"
          :class="[isFavorite ? 'btn btn-warning' : 'btn btn-success']"
        >
          {{ isFavorite ? 'Remove from' : 'Add to' }} Favorite
        </button>
      </div>
    </div>
    <span class="float-end small" v-if="ownername !== ''">
      *this contact info belongs to {{ ownername }}
    </span>
  </div>
</template>
<script setup>
const props = defineProps({
  name: {
    type: String,
    required: true,
  },
  phone: {
    type: Number,
    required: true,
  },
  ownername: String,
  email: {
    type: String,
    required: false,
    default: '-na-',
  },
  isFavorite: Boolean,
})

const emit = defineEmits(['update-favorite'])

function toggleFavorite() {
  emit('update-favorite', { isFavorite: props.isFavorite, name: props.name })
}
</script>
<style></style>

```

Project: Add New Contact component

Source Code:

Src/App.vue:

```
<template>
  <div class="bg-black text pt-3">
    <div class="container">
      <div class="text-white float-end">
        Contact Owner Name: <input type="text" v-model="ownerName" />
      </div>
      <AddContact></AddContact>
      <div class="row">
        <div class="col-12" v-for="contact in contacts" :key="contact.name">
          <Contact
            :name="contact.name"
            :phone="contact.phone"
            :ownername="contact.ownerName"
            :email="contact.email"
            :isFavorite="contact.isFavorite"
            @update-favorite="contact.isFavorite = onUpdateFavorite($event)"
          ></Contact>
        </div>
      </div>
    </div>
  </template>
<script setup>
import { ref, reactive } from 'vue'
import Contact from './components/Contact.vue'
import AddContact from './components/AddContact.vue'
const ownerName = ref('dotnetmastery')
const contacts = reactive([
  { name: 'Bhrugen', phone: 1234567890, ownerName: ownerName, isFavorite: false },
  { name: 'Bella', phone: 5555555555, ownerName: ownerName, isFavorite: true },
  {
    name: 'Ben',
    phone: 2222222222,
    ownerName: ownerName,
    email: 'ben@dotnetmastery.com',
    isFavorite: false,
  },
],
])

function onUpdateFavorite(oldValuesFromChildComponent) {
  console.log(oldValuesFromChildComponent.name)
  return !oldValuesFromChildComponent.isFavorite
}
</script>
<style></style>
```

Src/components/Contact.vue:

```
<template>
  <div class="bg-info rounded p-1 pb-4 m-1">
    <div class="row">
      <div class="col-6">
        <h3>Name: {{ name }}</h3>
        <p>Email: {{ email }}</p>
        <p>Phone: {{ phone }}</p>
      </div>
      <div class="col-3">
        <button
          @click="emit('update-favorite', { isFavorite: props.isFavorite, name: props.name })"
          :class="[isFavorite ? 'btn btn-warning' : 'btn btn-success']"
        >
          {{ isFavorite ? 'Remove from' : 'Add to' }} Favorite
        </button>
      </div>
    </div>
    <span class="float-end small" v-if="ownername !== ''">
      *this contact info belongs to {{ ownername }}
    </span>
  </div>
</template>
<script setup>
const props = defineProps({
  name: {
    type: String,
    required: true,
  },
  phone: {
    type: Number,
    required: true,
  },
  ownername: String,
  email: {
    type: String,
    required: false,
    default: '-na-',
  },
  isFavorite: Boolean,
})

const emit = defineEmits(['update-favorite'])
</script>
```

src/components/AddContact.vue:

```
<template>
  <div class="container border rounded p-2">
    <div class="text-center text-white">Add New Contact</div>
    <form @submit.prevent="addContact">
      <div class="row">
        <div class="col-4">
          <input type="text" class="form-control" v-model="contact.name" placeholder="Name" />
        </div>
        <div class="col-4">
```

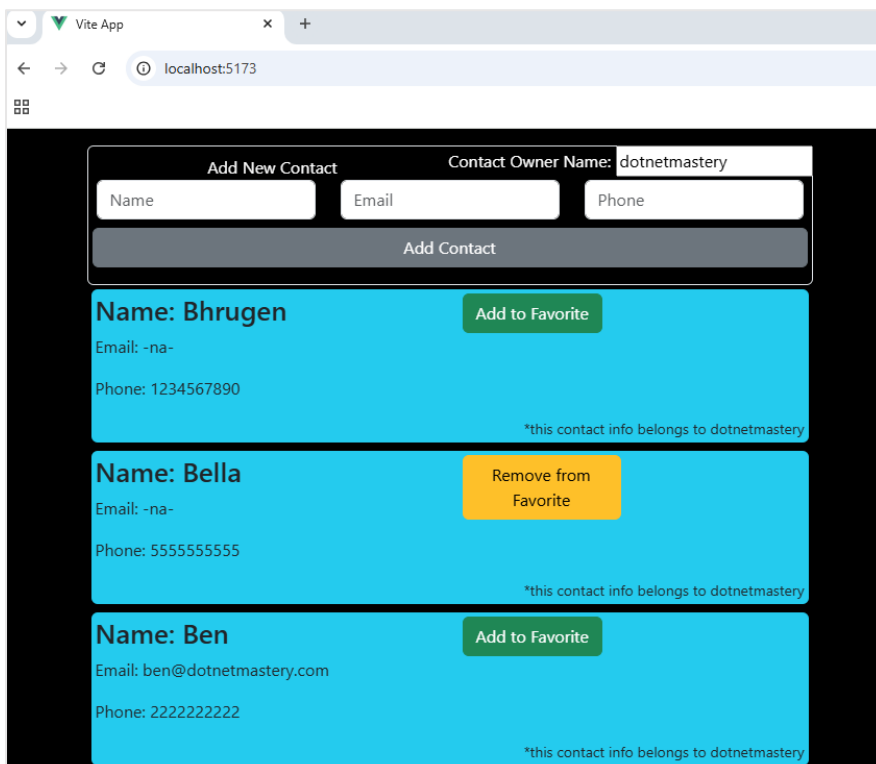
```

        <input type="text" class="form-control" v-model="contact.email" placeholder="Email" />
      </div>
      <div class="col-4">
        <input type="number" class="form-control" v-model="contact.phone" placeholder="Phone"
      />
    </div>
    <div class="col-6-offset-3 p-2">
      <button type="submit" class="btn btn-secondary w-100">Add Contact</button>
    </div>
  </div>
</form>
</div>
</template>
<script setup>
import { reactive } from 'vue'
const contact = reactive({
  name: '',
  email: '',
  phone: '',
})

function addContact() {
  console.log(contact)
  contact.name = ''
  contact.email = ''
  contact.phone = ''
}
</script>

```

Output:



Project: Passing props down to grandchild component

In Vue 3, passing props to a grandchild component involves a parent-child-grandchild hierarchy. Props are passed step by step, starting from the parent to the child, and then to the grandchild. Even if child component does not want the value it still must be passed down to child and then to grandchild.

Source Code:

Src/App.vue:

```
<template>
  <div class="bg-black text pt-3">
    <h1 class="text-center text-success">ContactOPedia</h1>
    <div class="container">
      <div class="row text-white p-2 mb-2">
        <div class="col-6">
          Owner Name: <input type="text" v-model="ownerName" class="form-control" />
        </div>
        <div class="col-6 text-end">
          Max Lucky Number: <input type="number" v-model.number="maxNumber" class="form-control" />
        </div>
      </div>
    </div>
    <br /><br />
    <AddContact></AddContact>
    <div class="row">
      <div class="col-12" v-for="contact in contacts" :key="contact.name">
        <Contact
          :name="contact.name"
          :phone="contact.phone"
          :ownername="contact.ownerName"
          :email="contact.email"
          :isFavorite="contact.isFavorite"
          :maxLuckyNumber="maxNumber"
          @update-favorite="contact.isFavorite = onUpdateFavorite($event)"
        ></Contact>
      </div>
    </div>
  </div>
</template>
<script setup>
import { ref, reactive } from 'vue'
import Contact from './components/Contact.vue'
import AddContact from './components/AddContact.vue'
const ownerName = ref('dotnetmastery')
const maxNumber = ref(100)
const contacts = reactive([
  { name: 'Bhrugen', phone: 1234567890, ownerName: ownerName, isFavorite: false },
  { name: 'Bella', phone: 5555555555, ownerName: ownerName, isFavorite: true },
  {
    name: 'Ben',
    phone: 2222222222,
    ownerName: ownerName,
    email: 'ben@dotnetmastery.com',
    isFavorite: false,
  },
],
])
```

```
function onUpdateFavorite(oldValuesFromChildComponent) {
  console.log(oldValuesFromChildComponent.name)
  return !oldValuesFromChildComponent.isFavorite
}
</script>
```

Src/components/Contact.vue:

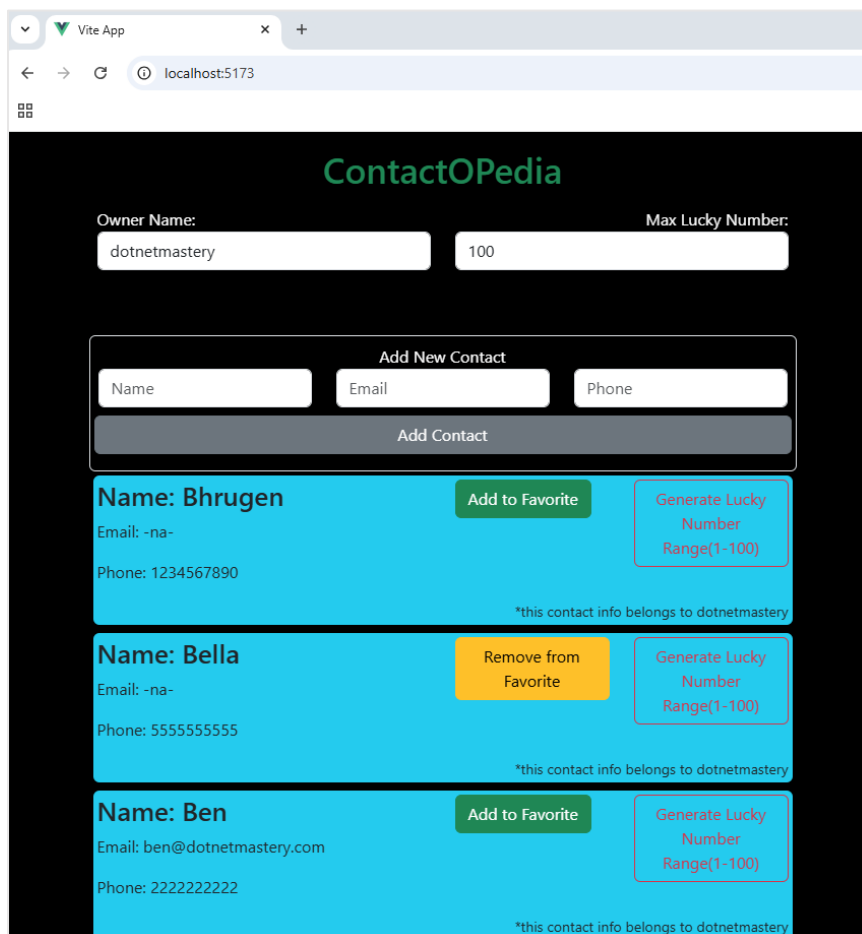
```
<template>
  <div class="bg-info rounded p-1 pb-4 m-1">
    <div class="row">
      <div class="col-6">
        <h3>Name: {{ name }}</h3>
        <p>Email: {{ email }}</p>
        <p>Phone: {{ phone }}</p>
      </div>
      <div class="col-3">
        <button
          @click="emit('update-favorite', { isFavorite: props.isFavorite, name: props.name })"
          :class="[isFavorite ? 'btn btn-warning' : 'btn btn-success']"
        >
          {{ isFavorite ? 'Remove from' : 'Add to' }} Favorite
        </button>
      </div>
      <div class="col-3">
        <LuckyNumber :maxNumber="maxLuckyNumber"></LuckyNumber>
      </div>
    </div>
    <span class="float-end small" v-if="ownername !== ''">
      *this contact info belongs to {{ ownername }}
    </span>
  </div>
</template>
<script setup>
import LuckyNumber from './LuckyNumber.vue'
const props = defineProps({
  name: {
    type: String,
    required: true,
  },
  phone: {
    type: Number,
    required: true,
  },
  ownername: String,
  email: {
    type: String,
    required: false,
    default: '-na-',
  },
  isFavorite: Boolean,
  maxLuckyNumber: Number,
})

const emit = defineEmits(['update-favorite'])
</script>
```

Src/components/LuckyNumber.vue:

```
<template>
  <div class="float-end">
    <button class="btn btn-outline-danger" @click="generateLuckyNumber">
      <span v-if="luckyNumber === 0">
        >Generate Lucky Number<br />
        Range(1-{{ maxNumber }})</span>
      <span v-else>Lucky Number: {{ luckyNumber }}<br />Click to generate new lucky number</span>
    </button>
  </div>
</template>
<script setup>
import { ref } from 'vue'
const luckyNumber = ref(0)
const props = defineProps({
  maxNumber: {
    type: Number,
    default: 10,
  },
})

function generateLuckyNumber() {
  luckyNumber.value = Math.floor(Math.random() * props.maxNumber)
  console.log(luckyNumber.value)
}
</script>
```



Project: Provide and Inject

Provide and inject enable dependency injection in Vue 3, allowing components to share data without passing props through every level of the component tree.

It's ideal for sharing global-like data across deeply nested components.

Src/App.vue:

```
<template>
  <div class="bg-black text pt-3">
    <h1 class="text-center text-success">ContactOPedia</h1>
    <div class="container">
      <div class="row text-white p-2 mb-2">
        <div class="col-6">
          Owner Name: <input type="text" v-model="ownerName" class="form-control" />
        </div>
        <div class="col-6 text-end">
          Max Lucky Number: <input type="number" v-model.number="maxNumber" class="form-control"
/>
        </div>
      </div>
      <br /><br />
      <AddContact></AddContact>
      <div class="row">
        <div class="col-12" v-for="contact in contacts" :key="contact.name">
          <Contact
            :name="contact.name"
            :phone="contact.phone"
            :ownername="contact.ownerName"
            :email="contact.email"
            :isFavorite="contact.isFavorite"
            @update-favorite="contact.isFavorite = onUpdateFavorite($event)"
          ></Contact>
        </div>
      </div>
    </div>
  </div>
</template>
<script setup>
import { ref, reactive, provide } from 'vue'
import Contact from './components/Contact.vue'
import AddContact from './components/AddContact.vue'
const ownerName = ref('dotnetmastery')
const maxNumber = ref(100)
provide('maxLuckyNumber', maxNumber)
const contacts = reactive([
  { name: 'Bhrugen', phone: 1234567890, ownerName: ownerName, isFavorite: false },
  { name: 'Bella', phone: 5555555555, ownerName: ownerName, isFavorite: true },
  { name: 'Ben', phone: 2222222222, ownerName: ownerName, email: 'ben@dotnetmastery.com',
    isFavorite: false },
])

function onUpdateFavorite(oldValuesFromChildComponent) {
  return !oldValuesFromChildComponent.isFavorite
}
</script>
```

```

<template>
  <div class="bg-info rounded p-1 pb-4 m-1">
    <div class="row">
      <div class="col-6">
        <h3>Name: {{ name }}</h3>
        <p>Email: {{ email }}</p>
        <p>Phone: {{ phone }}</p>
      </div>
      <div class="col-3">
        <button
          @click="emit('update-favorite', { isFavorite: props.isFavorite, name: props.name })"
          :class="[isFavorite ? 'btn btn-warning' : 'btn btn-success']"
        >
          {{ isFavorite ? 'Remove from' : 'Add to' }} Favorite
        </button>
      </div>
      <div class="col-3">
        <LuckyNumber></LuckyNumber>
      </div>
    </div>
    <span class="float-end small" v-if="ownername !== ''">
      *this contact info belongs to {{ ownername }}
    </span>
  </div>
</template>
<script setup>
import LuckyNumber from './LuckyNumber.vue'
const props = defineProps({
  name: {
    type: String,
    required: true,
  },
  phone: {
    type: Number,
    required: true,
  },
  ownername: String,
  email: {
    type: String,
    required: false,
    default: '-na-',
  },
  isFavorite: Boolean,
})

const emit = defineEmits(['update-favorite'])
</script>
<style></style>

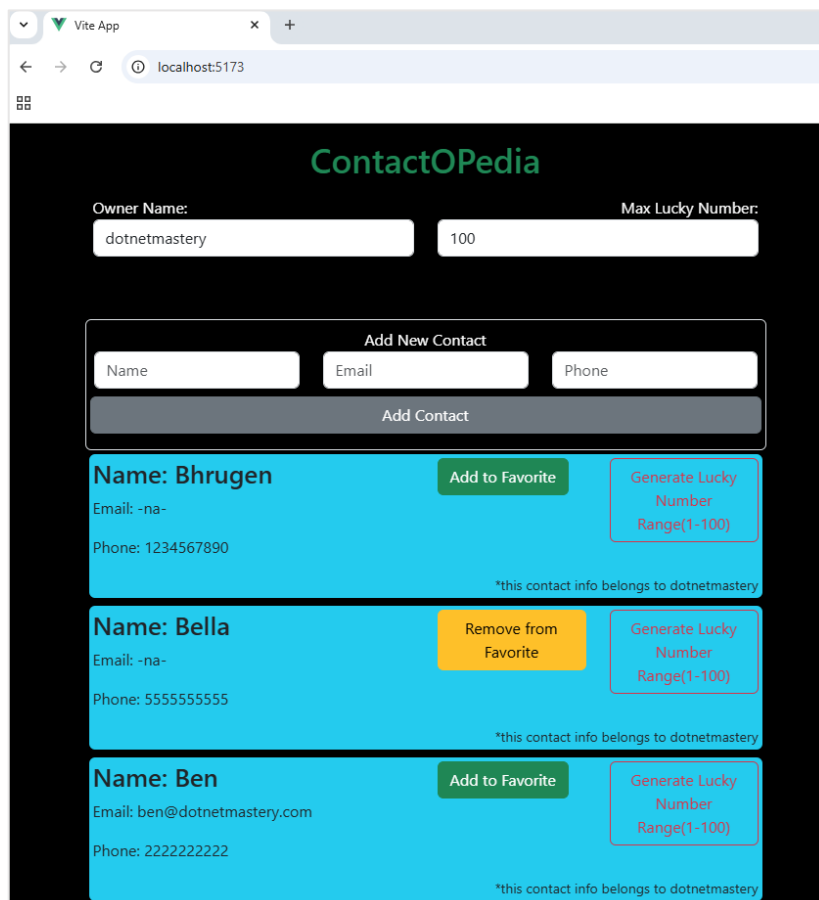
```

Src/components/LuckyNumber.vue:

```
<template>
  <div class="float-end">
    <button class="btn btn-outline-danger" @click="generateLuckyNumber">
      <span v-if="luckyNumber === 0">
        >Generate Lucky Number<br />
        Range(1-{{ maxNumber }})</span>
      <span v-else>Lucky Number: {{ luckyNumber }}<br />Click to generate new lucky number</span>
    </button>
  </div>
</template>
<script setup>
import { ref, inject } from 'vue'
const luckyNumber = ref(0)
const maxNumber = inject('maxLuckyNumber')

function generateLuckyNumber() {
  luckyNumber.value = Math.floor(Math.random() * maxNumber.value)
  console.log(luckyNumber.value)
}
</script>
<style></style>
```

Output:



Project: Dynamic Component

Dynamic components in Vue 3 allow you to swap components dynamically during runtime. This is particularly useful when creating reusable and flexible UI components. Vue offers the `<component>` element, which enables dynamic rendering based on the value of a specified property.

Src/App.vue:

```
<template>
  <div class="bg-black p-4">
    <div class="container">
      <button class="btn btn-primary text-black m-2" @click="newVersion = !newVersion">
        Toggle Component
      </button>
      <br />
      <button class="btn btn-primary text-black m-2" @click="newVersion = false">
        Lucky Number V1
      </button>
      <button class="btn btn-primary text-black m-2" @click="newVersion = true">
        Lucky Number V2
      </button>
      <br />
      <component :is="currentComponent" />
    </div>
  </div>
</template>
<script setup>
import { ref, computed } from 'vue'
import LuckyNumber from './components/LuckyNumber.vue'
import LuckyNumberV2 from './components/LuckyNumberV2.vue'
const newVersion = ref(false)
const currentComponent = computed(() => {
  return newVersion.value ? LuckyNumberV2 : LuckyNumber
})
</script>
```

Src/components/LuckyNumber.vue:

```
<template>
  <div>
    <button class="btn btn-danger text-black" @click="generateLuckyNumber">
      <span v-if="luckyNumber === 0">
        Generate Lucky Number<br />
        Range(1-100)</span>
      <span v-else>Lucky Number: {{ luckyNumber }}<br />Click to generate new lucky number</span>
    </button>
  </div>
</template>
<script setup>
import { ref } from 'vue'
const luckyNumber = ref(0)

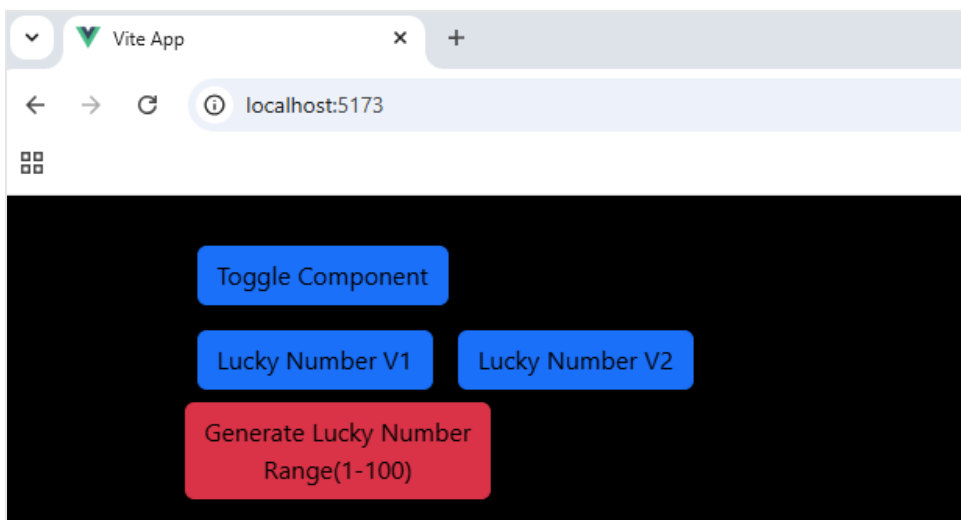
function generateLuckyNumber() {
  luckyNumber.value = Math.floor(Math.random() * 100)
  console.log(luckyNumber.value)
}
</script>
```

Src/components/LuckyNumberV2.vue:

```
<template>
  <div class="col-6 bg-white text-center p-4">
    <h4 class="text-success">Random Number Generator</h4>
    <hr />
    Max Range:
    <input v-model.number="maxNumber" class="rounded form-control" type="number" />
    <br />
    <button class="btn btn-danger text-black" @click="generateLuckyNumber">
      <span v-if="luckyNumber === 0"
        >Generate Lucky Number<br />
        Range(1-{{ maxNumber }})</span>
      <span v-else>Lucky Number: {{ luckyNumber }}<br />Click to generate new lucky number</span>
    </button>
  </div>
</template>
<script setup>
import { ref } from 'vue'
const luckyNumber = ref(0)
const maxNumber = ref(100)

function generateLuckyNumber() {
  luckyNumber.value = Math.floor(Math.random() * maxNumber.value)
  console.log(luckyNumber.value)
}
</script>
<style></style>
```

Output:



Project: Keep Alive

In Vue 3, <keep-alive> is a built-in component that is used to cache inactive components, preserving their state and avoiding re-rendering. It's particularly useful in scenarios like tab navigation or where you frequently switch between views but want to retain data or component states.

Source Code:

Src/App.vue:

```
<template>
  <div class="bg-black p-4">
    <div class="container">
      <button class="btn btn-primary text-black m-2" @click="newVersion = !newVersion">
        Toggle Component
      </button>
      <br />
      <button class="btn btn-primary text-black m-2" @click="newVersion = false">
        Lucky Number V1
      </button>
      <button class="btn btn-primary text-black m-2" @click="newVersion = true">
        Lucky Number V2
      </button>
      <br />
      <keep-alive :include="['LuckyNumber', 'LuckyNumberV2']">
        <component :is="currentComponent" />
      </keep-alive>
    </div>
  </div>
</template>
<script setup>
import { ref, computed } from 'vue'
import LuckyNumber from './components/LuckyNumber.vue'
import LuckyNumberV2 from './components/LuckyNumberV2.vue'
const newVersion = ref(false)
const currentComponent = computed(() => {
  return newVersion.value ? LuckyNumberV2 : LuckyNumber
})
</script>
```

Src/LuckyNumberV2.vue:

```
<template>
  <div class="col-6 bg-white text-center p-4">
    <h4 class="text-success">Random Number Generator</h4>
    <hr />
    Max Range:
    <input v-model.number="maxNumber" class="rounded form-control" type="number" />
    <br />
    <button class="btn btn-danger text-black" @click="generateLuckyNumber">
      <span v-if="luckyNumber === 0">
        >Generate Lucky Number<br />
        Range(1-{{ maxNumber }})</span>
      <span v-else>Lucky Number: {{ luckyNumber }}<br />Click to generate new lucky number</span>
    </button>
  </div>
```

```

</template>
<script setup>
import { ref } from 'vue'
const luckyNumber = ref(0)
const maxNumber = ref(100)

function generateLuckyNumber() {
  luckyNumber.value = Math.floor(Math.random() * maxNumber.value)
  console.log(luckyNumber.value)
}
</script>

```

Src/components/LuckyNumber.vue:

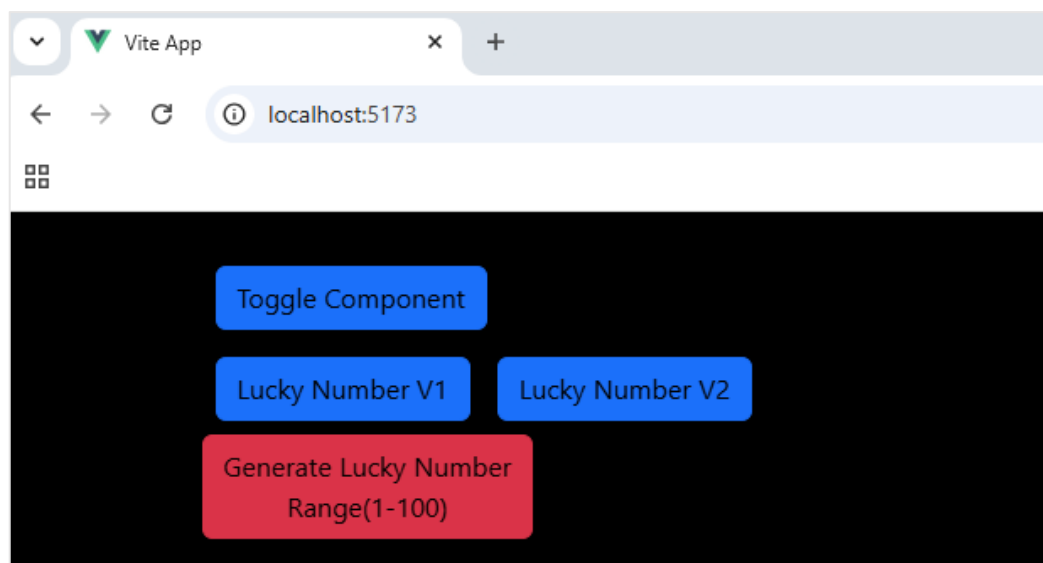
```

<template>
  <div>
    <button class="btn btn-danger text-black" @click="generateLuckyNumber">
      <span v-if="luckyNumber === 0">
        >Generate Lucky Number<br />
        Range(1-100)</span>
      <span v-else>Lucky Number: {{ luckyNumber }}<br />Click to generate new lucky number</span>
    </button>
  </div>
</template>
<script setup>
import { ref } from 'vue'
const luckyNumber = ref(0)

function generateLuckyNumber() {
  luckyNumber.value = Math.floor(Math.random() * 100)
  console.log(luckyNumber.value)
}
</script>

```

Output:



Project: Slots

In Vue 3, slots allow components to accept and render content passed from a parent component. They enable flexible and reusable components by letting developers customize the content of child components.

Source Code:

Src/App.vue:

```
<template>
  <LuckyNumberParentComponent>
    <p class="pt-2">We have two versions for picking lucky number</p>
    <p>Click the button to toggle between the two versions</p>
    <hr />
  </LuckyNumberParentComponent>
</template>
<script setup>
import LuckyNumberParentComponent from './components/LuckyNumberParentComponent.vue'
</script>
```

Src/components/LuckyNumberParentComponent.vue:

```
<template>
  <div class="bg-black p-4">
    <h1 class="text-center text-success">Learn Slots</h1>
    <div class="container text-center bg-white">
      <slot></slot>
      <button class="btn btn-primary text-black m-2" @click="newVersion = !newVersion">
        Toggle Component
      </button>
      <br />
      <button class="btn btn-primary text-black m-2" @click="newVersion = false">
        Lucky Number V1
      </button>
      <button class="btn btn-primary text-black m-2" @click="newVersion = true">
        Lucky Number V2
      </button>
      <br />
      <keep-alive :include="['LuckyNumber', 'LuckyNumberV2']">
        <component :is="currentComponent" class="border" />
      </keep-alive>
    </div>
  </div>
</template>
<script setup>
import { ref, computed } from 'vue'
import LuckyNumber from './LuckyNumber.vue'
import LuckyNumberV2 from './LuckyNumberV2.vue'
const newVersion = ref(false)
const currentComponent = computed(() => {
  return newVersion.value ? LuckyNumberV2 : LuckyNumber
})
</script>
<style></style>
```

Src/components/LuckyNumber.vue:

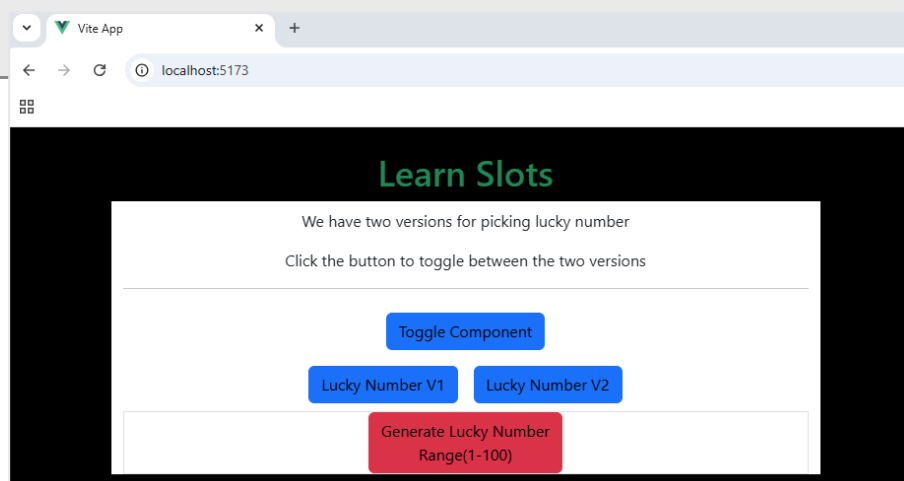
```
<template>
  <div>
    <button class="btn btn-danger text-black" @click="generateLuckyNumber">
      <span v-if="luckyNumber === 0">Generate Lucky Number<br />
        Range(1-100)</span>
      <span v-else>Lucky Number: {{ luckyNumber }}<br />Click to generate new lucky number</span>
    </button>
  </div>
</template>
<script setup>
import { ref } from 'vue'
const luckyNumber = ref(0)

function generateLuckyNumber() {
  luckyNumber.value = Math.floor(Math.random() * 100)
}
</script>
```

Src/components/LuckyNumberV2.vue:

```
<template>
  <div class="col-6 bg-white text-center p-4">
    <h4 class="text-success">Random Number Generator</h4>
    <hr />
    Max Range:
    <input v-model.number="maxNumber" class="rounded form-control" type="number" />
    <br />
    <button class="btn btn-danger text-black" @click="generateLuckyNumber">
      <span v-if="luckyNumber === 0">Generate Lucky Number<br />
        Range(1-{{ maxNumber }})</span>
      <span v-else>Lucky Number: {{ luckyNumber }}<br />Click to generate new lucky number</span>
    </button>
  </div>
</template>
<script setup>
import { ref } from 'vue'
const luckyNumber = ref(0)
const maxNumber = ref(100)

function generateLuckyNumber() {
  luckyNumber.value = Math.floor(Math.random() * maxNumber.value)
}
</script>
```



Project: Named Slots

Named slots in Vue 3 allow you to define specific sections within a component that can be filled with content from the parent component. This provides greater flexibility and organization when designing reusable components.

Src/App.vue:

```
<template>
  <LuckyNumberParentComponent>
    <template v-slot:default>
      <p class="pt-2">We have two versions for picking lucky number</p>
      <hr />
    </template>
    <template v-slot:moreInfo>
      <p>Click the button to toggle between the two versions</p>
    </template>
    <template v-slot:learnSlot>
      <button class="btn btn-primary" @click="showMessage">What will we learn?</button>
      <h4 class="text-success">{{ message }}</h4>
    </template>
  </LuckyNumberParentComponent>
</template>
<script setup>
import { ref } from 'vue'
import LuckyNumberParentComponent from './components/LuckyNumberParentComponent.vue'
const message = ref('')
function showMessage() {
  message.value = 'We will learn how to use slots'
}
</script>
```

Src/components/LuckyNumberParentComponent.vue:

```
<template>
  <div class="bg-black p-4">
    <h1 class="text-center text-success">Learn Slots</h1>
    <div class="container text-center bg-white">
      <slot></slot>
      <button class="btn btn-primary text-black m-2" @click="newVersion = !newVersion">
        Toggle Component
      </button>
      <br />
      <slot name="moreInfo"></slot>
      <button class="btn btn-primary text-black m-2" @click="newVersion = false">
        Lucky Number V1
      </button>
      <button class="btn btn-primary text-black m-2" @click="newVersion = true">
        Lucky Number V2
      </button>
      <br />
      <slot name="learnSlot"></slot>
      <keep-alive :include="['LuckyNumber', 'LuckyNumberV2']">
        <component :is="currentComponent" class="border" />
      </keep-alive>
    </div>
  </div>
</template>
```

```

<script setup>
import { ref, computed } from 'vue'
import LuckyNumber from './LuckyNumber.vue'
import LuckyNumberV2 from './LuckyNumberV2.vue'
const newVersion = ref(false)
const currentComponent = computed(() => {
  return newVersion.value ? LuckyNumberV2 : LuckyNumber
})
</script>
<style></style>

```

Src/components/LuckyNumber.vue:

```

<template>
  <div>
    <button class="btn btn-danger text-black" @click="generateLuckyNumber">
      <span v-if="luckyNumber === 0">
        >Generate Lucky Number<br />
        Range(1-100)</span>
      <span v-else>Lucky Number: {{ luckyNumber }}<br />Click to generate new lucky number</span>
    </button>
  </div>
</template>
<script setup>
import { ref } from 'vue'
const luckyNumber = ref(0)

function generateLuckyNumber() {
  luckyNumber.value = Math.floor(Math.random() * 100)
}
</script>
<style></style>

```

Src/components/LuckyNumberV2.vue:

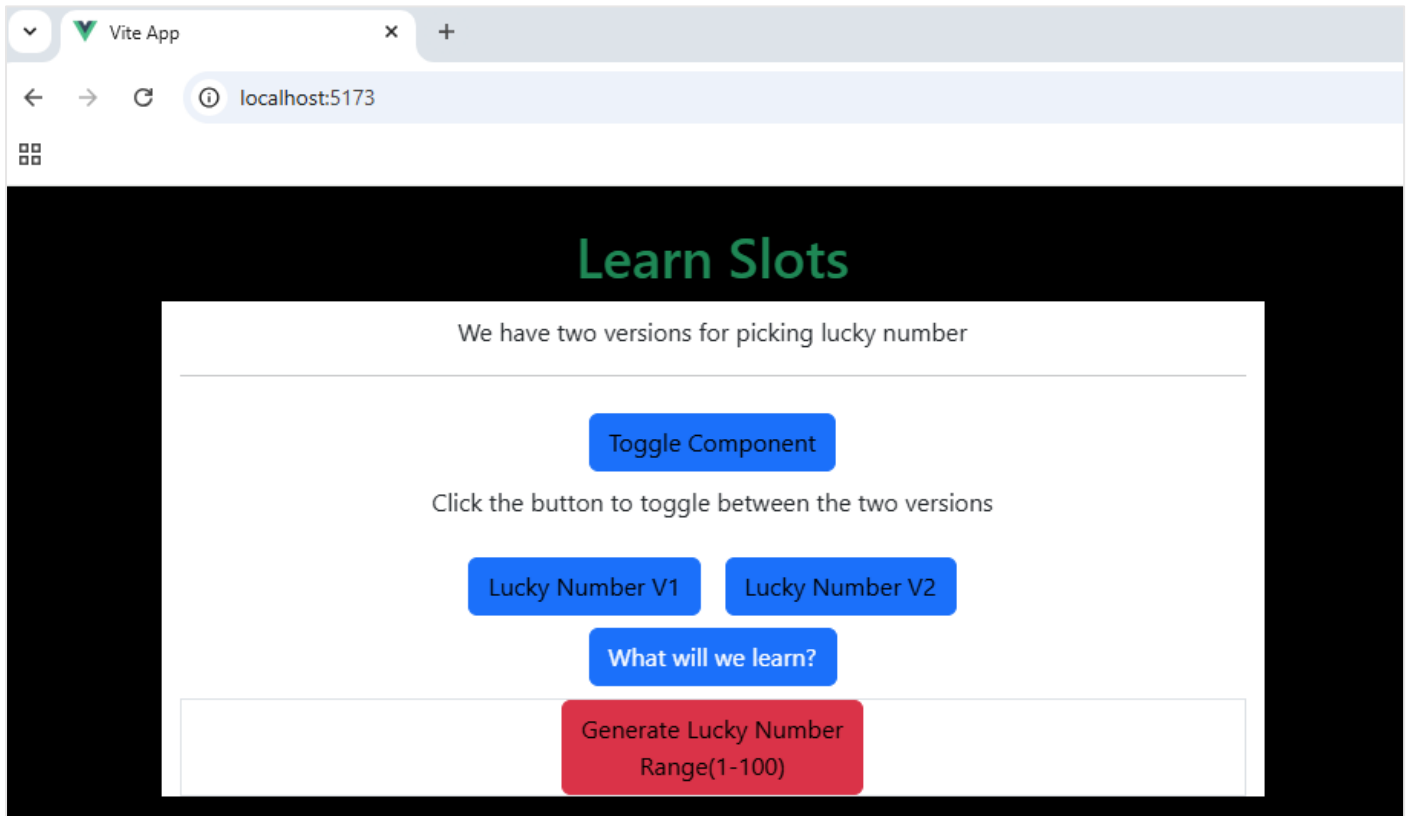
```

<template>
  <div class="col-6 bg-white text-center p-4">
    <h4 class="text-success">Random Number Generator</h4>
    <hr />
    Max Range:
    <input v-model.number="maxNumber" class="rounded form-control" type="number" />
    <br />
    <button class="btn btn-danger text-black" @click="generateLuckyNumber">
      <span v-if="luckyNumber === 0">
        >Generate Lucky Number<br />
        Range(1-{{ maxNumber }})</span>
      <span v-else>Lucky Number: {{ luckyNumber }}<br />Click to generate new lucky number</span>
    </button>
  </div>
</template>
<script setup>
import { ref } from 'vue'
const luckyNumber = ref(0)
const maxNumber = ref(100)

```

```
function generateLuckyNumber() {  
  luckyNumber.value = Math.floor(Math.random() * maxNumber.value)  
  console.log(luckyNumber.value)  
}  
</script>
```

Output:



Project: Scoped Styles

In Vue 3, scoped styles ensure that the CSS rules defined in a component apply only to that specific component, preventing style conflicts across the application.

Source Code:

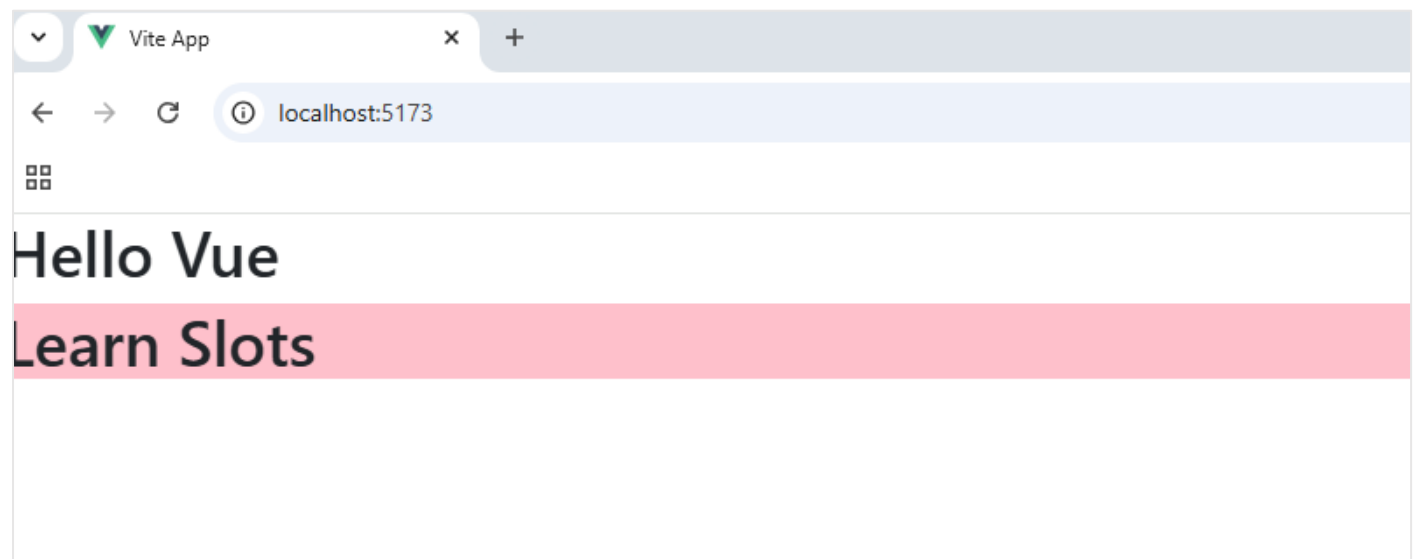
Src/App.vue:

```
<template>
  <h1>Hello Vue</h1>
  <LuckyNumberParentComponent> </LuckyNumberParentComponent>
</template>
<script setup>
import LuckyNumberParentComponent from './components/LuckyNumberParentComponent.vue'
</script>
```

Src/components/LuckyNumberParentComponent.vue:

```
<template>
  <h1>Learn Slots</h1>
</template>
<script setup></script>
<style scoped>
h1 {
  background-color: pink;
}
</style>
```

Output:



Project: Lifecycle Demo

The component lifecycle in Vue 3 consists of a series of stages that a component goes through from creation to destruction. Vue provides lifecycle hooks to let developers execute specific actions at each stage.

Source Code:

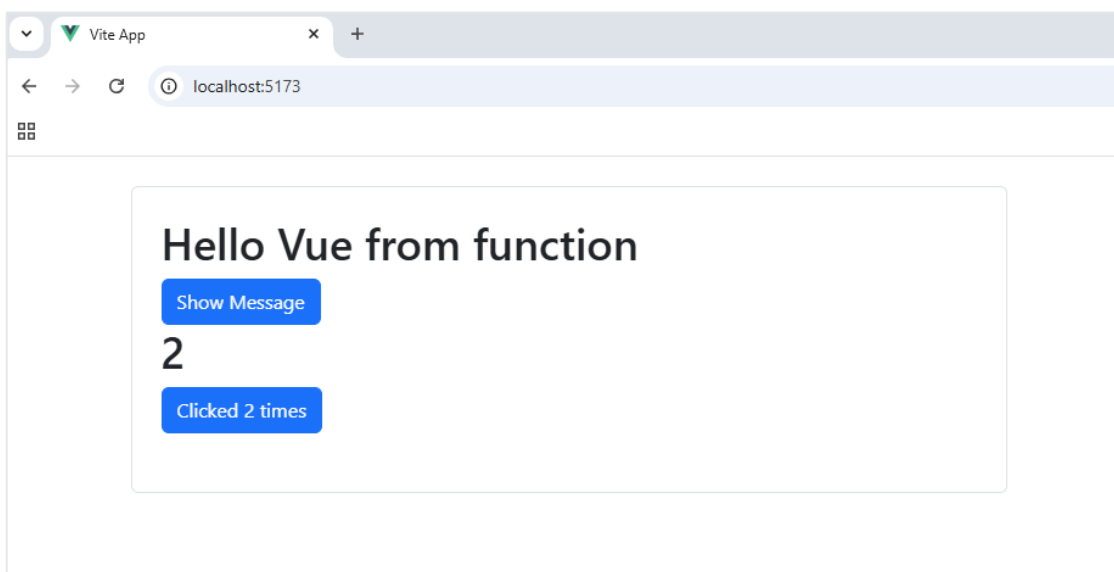
Src/App.vue:

```
<template>
  <div class="container p-4 border rounded mt-4">
    <h1>{{ message }}</h1>
    <button @click="showMessage" class="btn btn-primary">Show Message</button>
    <br />
    <h1>{{ count }}</h1>
    <button @click="count++" class="btn btn-primary">Clicked {{ count }} times</button>
    <br />
    <br />
    <ButtonCounter v-if="count == 1"></ButtonCounter>
  </div>
</template>
<script setup>
import {
  ref,
  onBeforeMount,
  onMounted,
  onBeforeUpdate,
  onUpdated,
  onBeforeUnmount,
  onUnmounted,
} from 'vue'
import ButtonCounter from './components/ButtonCounter.vue'
const message = ref('Hello Vue')
const count = ref(0)
function showMessage() {
  message.value = 'Hello Vue from function'
}
onBeforeMount(() => {
  console.log('onBeforeMount - App.vue')
})
onMounted(() => {
  console.log('onMounted - App.vue')
})
onBeforeUpdate(() => {
  console.log('onBeforeUpdate - App.vue')
})
onUpdated(() => {
  console.log('onUpdated - App.vue')
})
onBeforeUnmount(() => {
  console.log('onBeforeUnmount - App.vue')
})
onUnmounted(() => {
  console.log('onUnmounted - App.vue')
})
</script>
```

Src/components/ButtonCounter.vue:

```
<template>
  <button @click="count++" class="btn btn-primary">Clicked {{ count }} times</button>
</template>
<script setup>
import {
  ref,
  onBeforeMount,
  onMounted,
  onBeforeUpdate,
  onUpdated,
  onBeforeUnmount,
  onUnmounted,
} from 'vue'
const count = ref(0)
onBeforeMount(() => {
  console.log('onBeforeMount - ButtonCounter.vue')
})
onMounted(() => {
  console.log('onMounted - ButtonCounter.vue')
})
onBeforeUpdate(() => {
  console.log('onBeforeUpdate - ButtonCounter.vue')
})
onUpdated(() => {
  console.log('onUpdated - ButtonCounter.vue')
})
onBeforeUnmount(() => {
  console.log('onBeforeUnmount - ButtonCounter.vue')
})
onUnmounted(() => {
  console.log('onUnmounted - ButtonCounter.vue')
})
</script>
<style></style>
```

Output:



Project: Composable in Vue

In Vue 3, composables are reusable functions that encapsulate reactive state and logic, enabling better code organization and reuse in your applications. They are a core part of the Composition API.

Source Code:

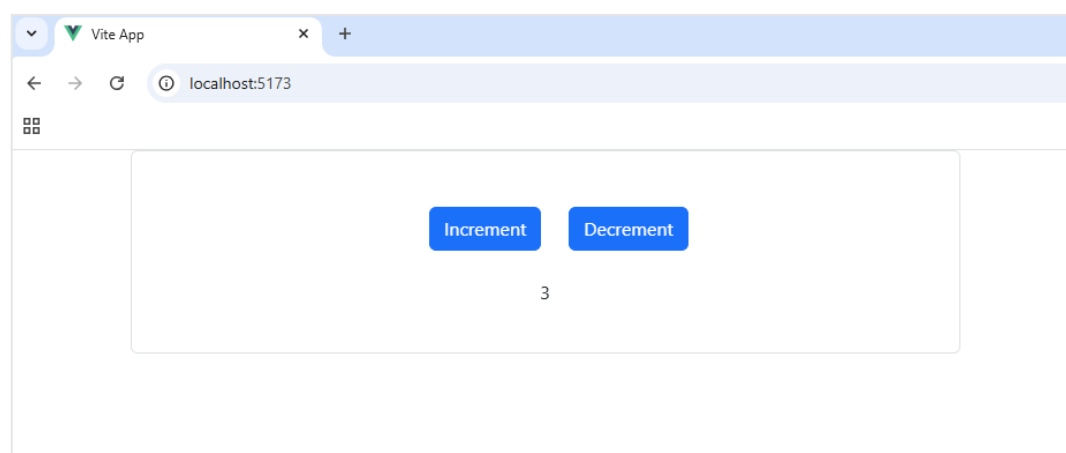
Src/App.vue:

```
<template>
  <div class="container p-4 border rounded text-center">
    <button @click="increment" class="btn btn-primary m-4">Increment</button>
    <button @click="decrement" class="btn btn-primary">Decrement</button>
    <p>{{ count }}</p>
  </div>
</template>
<script setup>
import { useCounter } from './composables/useCounter'
const { count, increment, decrement } = useCounter()
</script>
<style></style>
```

Src/composables/useCounter.js:

```
import { ref } from 'vue'
export function useCounter() {
  const count = ref(0)
  function increment() {
    count.value++
  }
  function decrement() {
    count.value--
  }
  return { count, increment, decrement }
}
```

Output:



Project: Props in Composable

Passing an initial value to composables in Vue 3 allows you to customize their behavior when called. You can achieve this by using parameters in the composable function.

Src/App.vue:

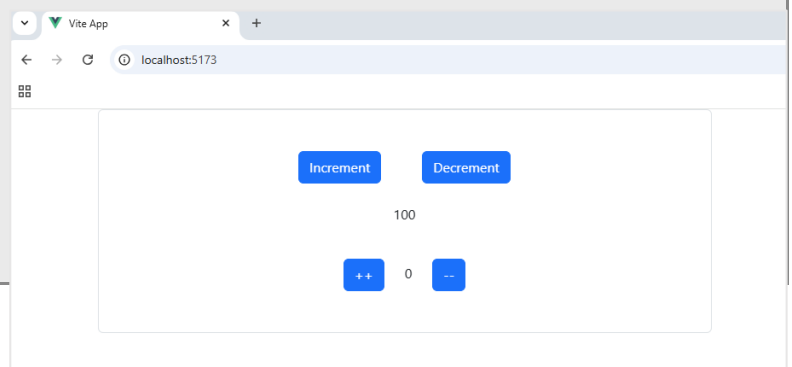
```
<template>
  <div class="container p-4 border rounded text-center">
    <button @click="increment" class="btn btn-primary m-4">Increment</button>
    <button @click="decrement" class="btn btn-primary m-4">Decrement</button>
    <p>{{ count }}</p>
    <ButtonCounter></ButtonCounter>
  </div>
</template>
<script setup>
import ButtonCounter from './components/ButtonCounter.vue'
import { useCounter } from './composables/useCounter'
const { count, increment, decrement } = useCounter(100, 10)
</script>
<style></style>
```

Src/components/ButtonCounter.vue:

```
<template>
  <button @click="increment" class="btn btn-primary m-4">++</button>
  <span>{{ count }}</span>
  <button @click="decrement" class="btn btn-primary m-4">--</button>
</template>
<script setup>
import { useCounter } from '@composables/useCounter'
const { count, increment, decrement } = useCounter()
</script>
<style></style>
```

Src/composables/useCounter.js:

```
import { ref } from 'vue'
export function useCounter(initialValue = 0, step = 1) {
  const count = ref(initialValue)
  function increment() {
    count.value += step
  }
  function decrement() {
    count.value -= step
  }
  return { count, increment, decrement }
}
```



Project: Routing in action

To add routes in Vue Router, create an array of route objects, each defining the path and the component it should render. We will have to install "vue-router" package.

Src/components/Contact.vue

```
<template>
  <div>Contact Infomration</div>
</template>
```

Src/components/HomePage.vue

```
<template>
  <div>Home Page </div>
</template>
```

Src/router/routes.js:

We will configure routing inside routes.js where we will register path and the component to render on that path

```
import HomePage from "@/components/HomePage.vue";
import Contact from "@/components/Contact.vue";
import { createRouter, createWebHistory } from "vue-router";

const router = createRouter({
  history: createWebHistory(),
  routes: [
    { path: "/", component: HomePage },
    { path: "/contact", component: Contact },
  ],
});

export default router;
```

Src/main.js:

We will import the routing configuration from routes.js and mount that on our application to add router

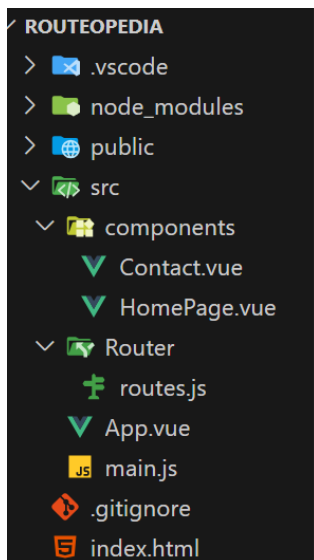
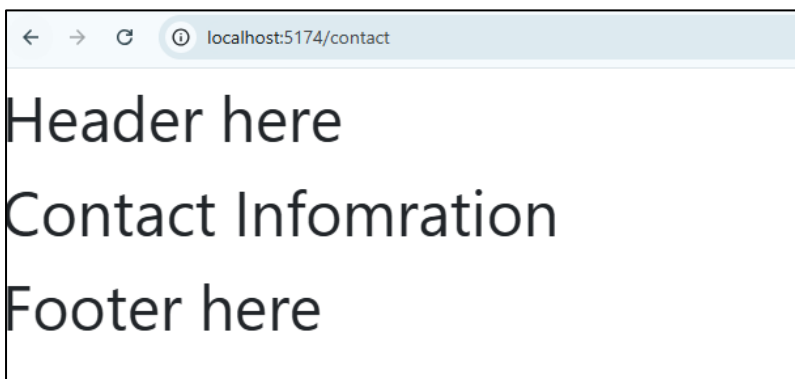
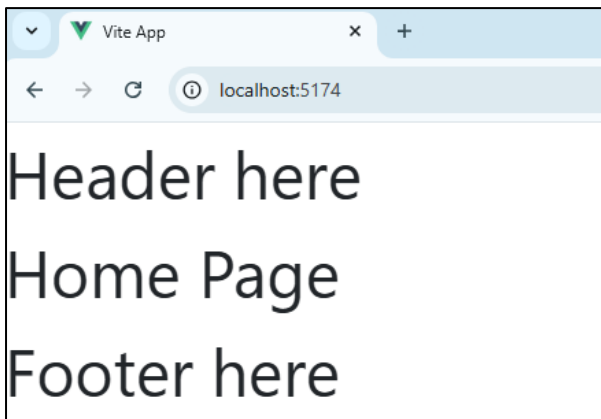
```
import { createApp } from "vue";
import App from "./App.vue";
import router from "./Router/routes.js";
const app = createApp(App);

app.use(router);
app.mount("#app");
```

src/App.vue:

```
<template>
  <div>Header here</div>
  <router-view></router-view> // all the components will be mounted here from router
  <div>Footer here</div>
</template>
```

Output:



Project: Router Link

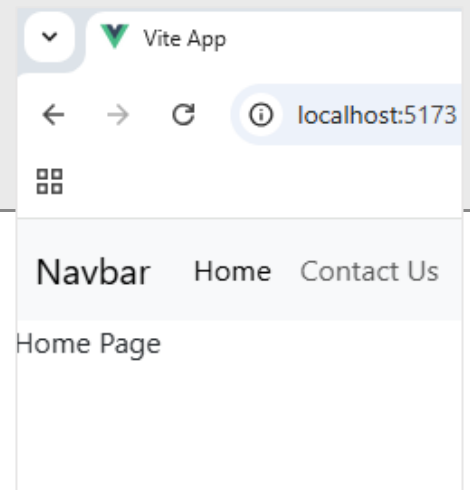
<router-link> is a Vue Router component used to create navigation links in Vue applications. It automatically handles routing and updates the URL without reloading the page, enabling a seamless Single Page Application (SPA) experience.

Src/App.vue:

```
<template>
  <Header></Header>
  <router-view></router-view>
</template>
<script setup>
import Header from './components/Layout/Header.vue'
</script>
```

Src/components/Layout/Header.vue:

```
<template>
  <nav class="navbar navbar-expand-lg bg-body-tertiary">
    <div class="container-fluid">
      <a class="navbar-brand" href="#">Navbar</a>
      <button
        class="navbar-toggler"
        type="button"
        data-bs-toggle="collapse"
        data-bs-target="#navbarSupportedContent"
        aria-controls="navbarSupportedContent"
        aria-expanded="false"
        aria-label="Toggle navigation"
      >
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="navbarSupportedContent">
        <ul class="navbar-nav me-auto mb-2 mb-lg-0">
          <li class="nav-item">
            <router-link class="nav-link active" aria-current="page" to="/">Home</router-link>
          </li>
          <li class="nav-item">
            <router-link class="nav-link" aria-current="page" to="/contact">Contact Us</router-link>
          </li>
        </ul>
        <form class="d-flex" role="search">
          <a class="nav-link disabled" aria-disabled="true">Login</a>
        </form>
      </div>
    </div>
  </nav>
</template>
```



Project: Dynamic Route Binding

Dynamic routes in Vue 3 Router allow you to define routes with placeholders for parameters, enabling flexibility in handling paths with variable segments. Use the ":" syntax to define parameterized paths.

Source Code:

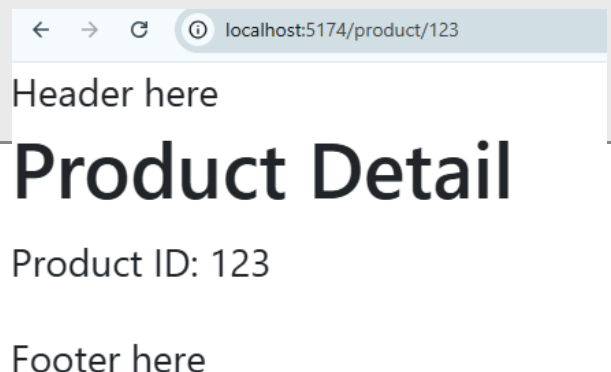
Src/router/routes.js:

```
import { createRouter, createWebHistory } from 'vue-router'
import HomePage from '@/components/Home/HomePage.vue'
import Contact from '@/components/Home/Contact.vue'
import ProductList from '@/components/Product/ProductList.vue'
import ProductDetail from '@/components/Product/ProductDetail.vue'

const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: '/',
      component: HomePage,
    },
    {
      path: '/contact',
      component: Contact,
    },
    {
      path: '/product/:productId',
      component: ProductDetail,
    },
  ],
})
export default router
```

Src/components/Product/ProductDetail.vue:

```
<template>
  <div>
    <h1>Product Detail</h1>
    <p>Product ID: {{ route.params.productId }}</p>
  </div>
</template>
<script setup>
import { useRoute } from 'vue-router'
const route = useRoute()
console.log(route.params)
</script>
```



Project: Multiple parameters in URL:

In Vue Router, you can define routes with multiple parameters in the URL by specifying them using the ":" syntax.

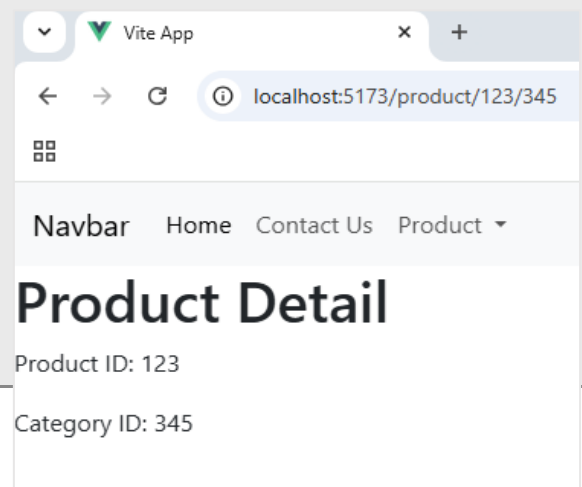
Src/router/routes.js:

```
import { createRouter, createWebHistory } from 'vue-router'
import HomePage from '@components/Home/HomePage.vue'
import Contact from '@components/Home/Contact.vue'
import ProductList from '@components/Product/ProductList.vue'
import ProductDetail from '@components/Product/ProductDetail.vue'

const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: '/',
      component: HomePage,
    },
    {
      path: '/product/:productId/:categoryId?',
      component: ProductDetail,
    },
    {
      path: '/product',
      component: ProductDetail,
    },
  ],
})
export default router
```

Src/components/Product/ProductDetail.vue:

```
<template>
  <div>
    <h1>Product Detail</h1>
    <div v-if="route.params.productId">
      <p>Product ID: {{ route.params.productId }}</p>
    </div>
    <div v-else>
      <p>No product ID found</p>
    </div>
    <div v-if="route.params.categoryId">
      <p>Category ID: {{ route.params.categoryId }}</p>
    </div>
  </div>
</template>
<script setup>
import { useRoute } from 'vue-router'
const route = useRoute()
</script>
```



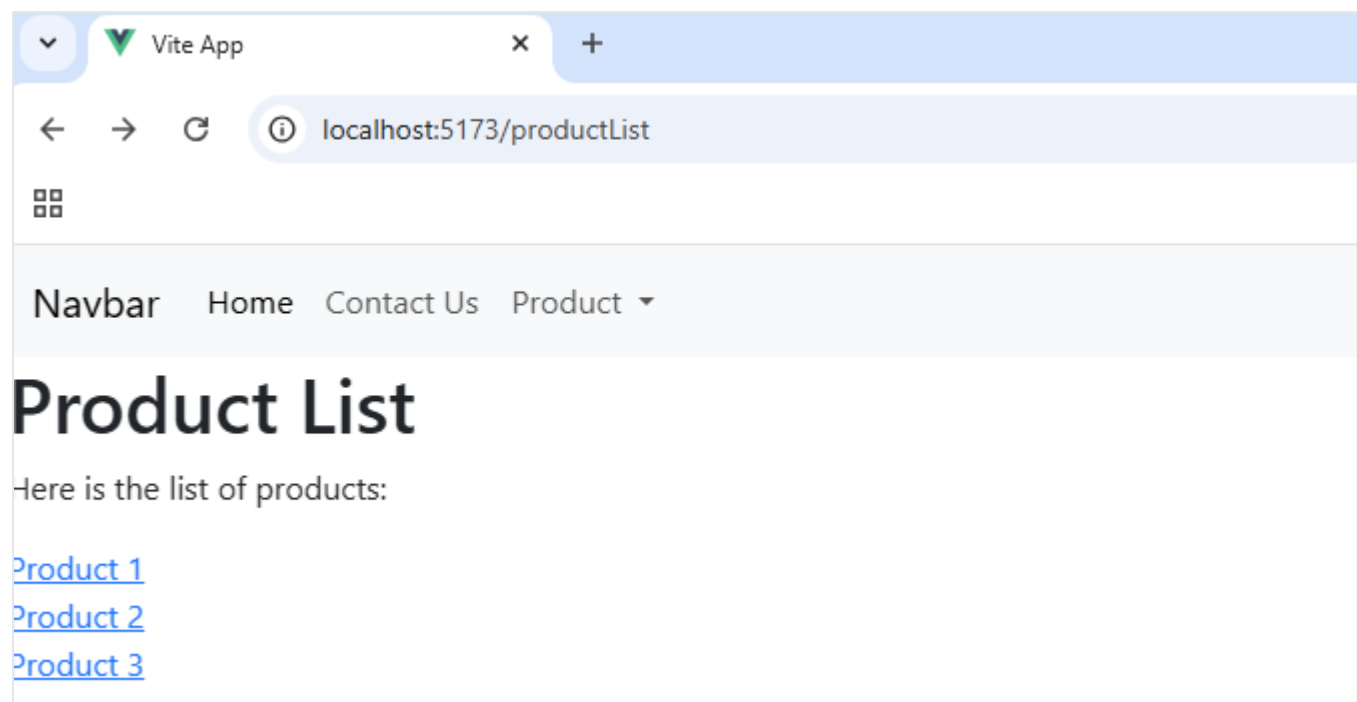
Project: Route with dynamic values

Source Code:

Src/components/Product/ProductList.vue:

```
<template>
  <div>
    <h1>Product List</h1>
    <p>Here is the list of products:</p>
    <div v-for="product in productList" :key="product.id">
      <router-link :to="`/product/${product.id}`">{{ product.name }}</router-link>
    </div>
  </div>
</template>
<script setup>
const productList = [
  {
    id: 1,
    name: 'Product 1',
  },
  {
    id: 2,
    name: 'Product 2',
  },
  {
    id: 3,
    name: 'Product 3',
  },
]
</script>
```

Output:



Project: Named routes

Named routes in Vue Router allow you to assign a specific name to a route, making navigation and route management more convenient and readable, especially when dealing with complex apps.

Source Code:

Src/router/routes.js:

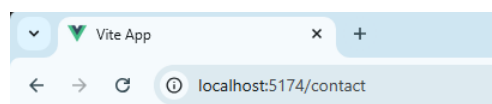
```
import HomePage from "@/components/HomePage.vue";
import Contact from "@/components/Contact.vue";
import ProductDetail from "@/components/ProductDetail.vue";
import { createRouter, createWebHistory } from "vue-router";

const router = createRouter({
  history: createWebHistory(),
  routes: [
    { path: "/", component: HomePage, name: "home" },
    { path: "/contact", component: Contact, name: "contact" },
  ],
});

export default router;
```

src/app.vue

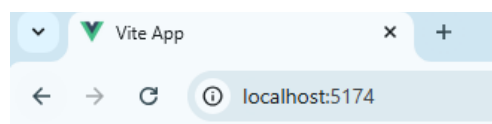
```
<template>
  <router-link :to="{ name: 'home' }">Home</router-link>
  <router-link :to="{ name: 'contact' }">Contact</router-link>
  <router-view></router-view>
  <div>Footer here</div>
</template>
```



[Home](#) [Contact](#)

Contact Infomration

Footer here



[Home](#) [Contact](#)

Home Page

Footer here

Project: Redirect URL

Redirects in Vue Router allow you to automatically direct users from one route to another.

If someone goes to `/contact`, in the example below it will redirect to named route “contact” which will take them to `/contact-us` route

Source Code:

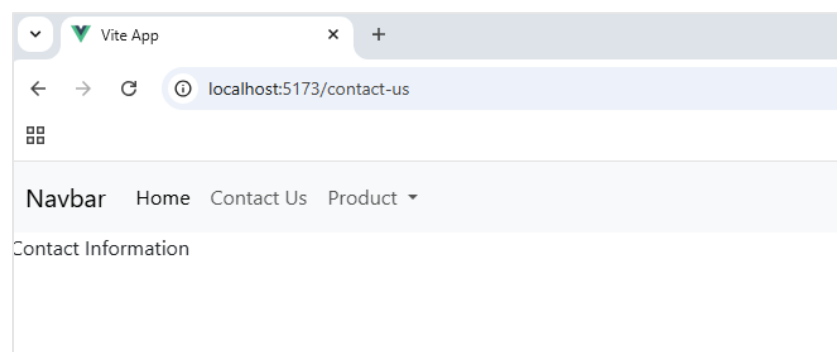
Src/router/routes.js:

```
import { createRouter, createWebHistory } from 'vue-router'
import HomePage from '@components/Home/HomePage.vue'
import Contact from '@components/Home/Contact.vue'
import ProductList from '@components/Product/ProductList.vue'
import ProductDetail from '@components/Product/ProductDetail.vue'

const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: '/',
      component: HomePage,
    },
    {
      path: '/contact',
      redirect: { name: 'contact' },
    },
    {
      path: '/contact-us',
      component: Contact,
      name: 'contact',
    },
  ],
})

export default router
```

Output:



Project: Not Found

In Vue Router, handling "Not Found" pages (for undefined routes) is straightforward. You can use a catch-all route.

Source Code:

Src/router/routes.js

```
import { createRouter, createWebHistory } from 'vue-router'
import HomePage from '@components/Home/HomePage.vue'
import Contact from '@components/Home/Contact.vue'
import ProductList from '@components/Product/ProductList.vue'
import ProductDetail from '@components/Product/ProductDetail.vue'
import NotFound from '@components/Layout/NotFound.vue'

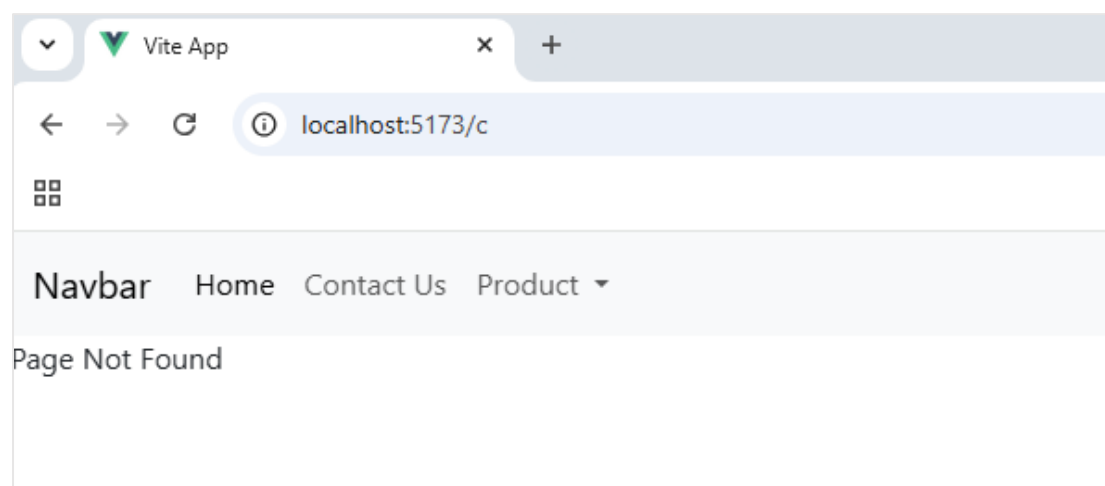
const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: '/',
      component: HomePage,
    },
    {
      path: '/*',
      component: NotFound,
    },
  ],
})

export default router
```

src/components/Layout/NotFound.vue:

```
<template>
  <div>Page Not Found</div>
</template>
```

Output:



Project: Navigate using script

Router.push is a Vue Router method used to programmatically navigate to a specific route in a Vue application.

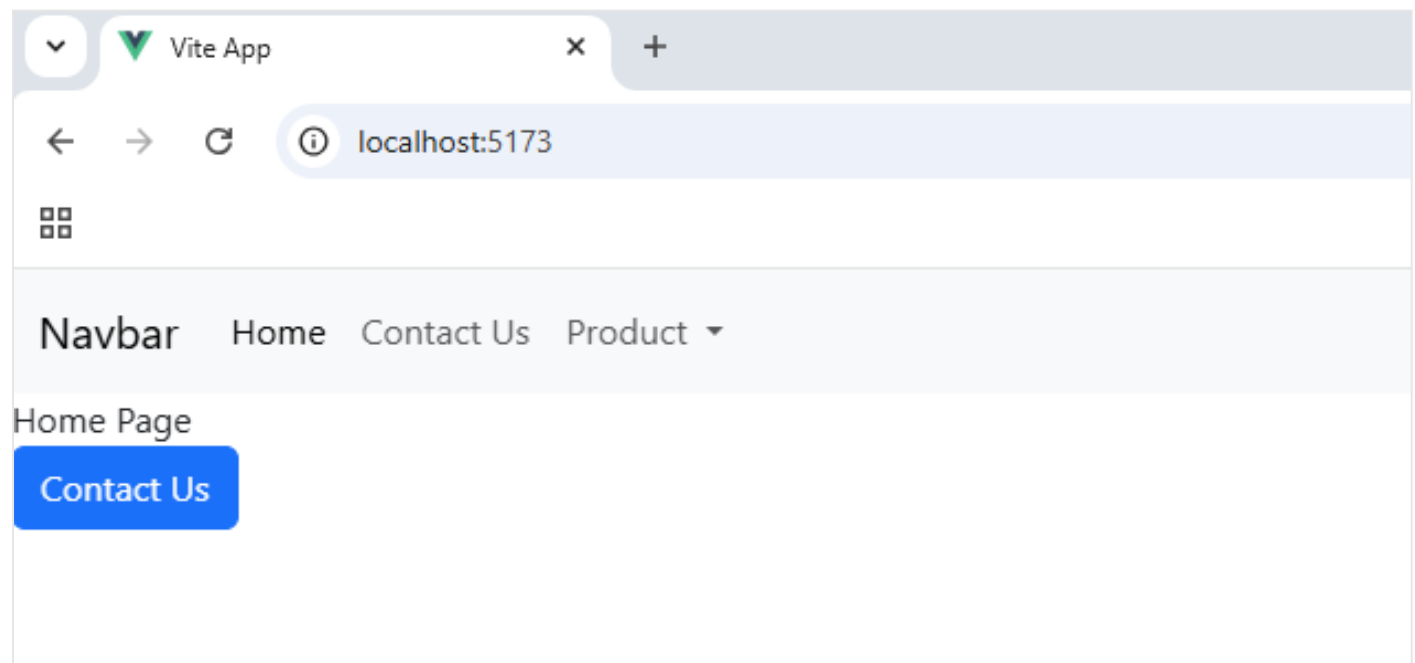
Source Code:

Src/components/Home/HomePage.vue:

```
<template>
  <div>Home Page</div>
  <button @click="goToContact" class="btn btn-primary">Contact Us</button>
</template>
<script setup>
import { useRouter } from 'vue-router'
const router = useRouter()

function goToContact() {
  // router.push({ name: 'contact' })
  router.push('/contact-us')
}
</script>
```

Output:



Project: Back Button

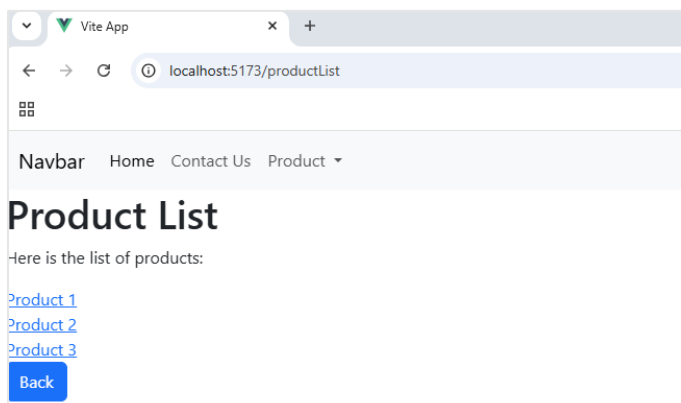
Router.go() is a Vue Router method that allows you to navigate through the browser's history programmatically.

Source Code:

Src/components/Product/ProductList.vue:

```
<template>
  <div>
    <h1>Product List</h1>
    <p>Here is the list of products:</p>
    <div v-for="product in productList" :key="product.id">
      <router-link
        :to="{ name: 'productDetails', params: { productId: product.id, categoryId: 777 } }"
        >{{ product.name }}</router-link>
    </div>
    <button @click="$router.go(-1)" class="btn btn-primary">Back</button>
  </div>
</template>
<script setup>
const productList = [
  {
    id: 1,
    name: 'Product 1',
  },
  {
    id: 2,
    name: 'Product 2',
  },
  {
    id: 3,
    name: 'Product 3',
  },
]
</script>
```

Output:



Project: Protected pages from unauthenticated access

Global navigation guards in Vue 3 are used to control and manage route navigation across the entire application. They allow you to execute logic before, during, or after a route change.

Protected routes in Vue Router ensure that only authorized users can access certain routes in your application. Typically, this is done by checking authentication or user permissions before granting access.

Use `router.beforeEach()` global navigation guard to check user authentication or permissions before navigating.

Source Code:

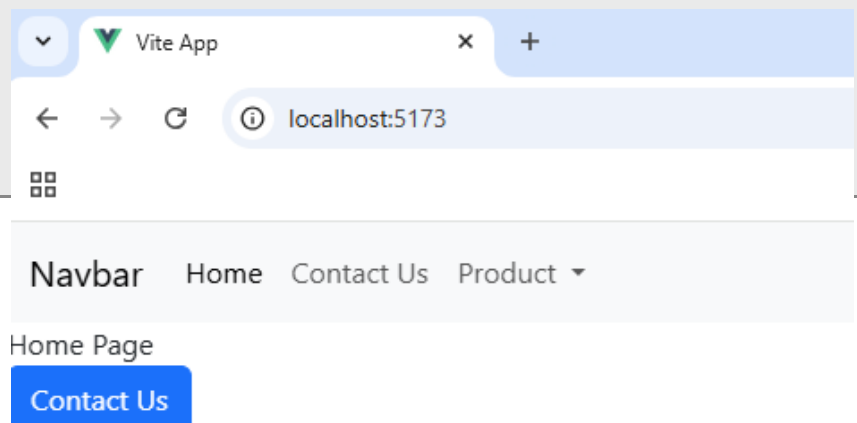
```
import { createRouter, createWebHistory } from 'vue-router'
import HomePage from '@/components/Home/HomePage.vue'
import Contact from '@/components/Home/Contact.vue'
import Login from '@/components/Authentication/Login.vue'

const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: '/',
      component: HomePage, name: 'home',
    },
    {
      path: '/contact-us',
      component: Contact, name: 'contact',
    },
    {
      path: '/login',
      component: Login, name: 'login',
    },
  ],
})

router.beforeEach((to, from) => {
  console.log('Global Before Each')
  console.log(to, from)
  const isAuthenticated = false

  if (to.name === 'home') {
    return true
  }
  if (!isAuthenticated && to.name !== 'login') {
    return { name: 'login' }
  }
  return true
})

export default router
```



Project: Per Route Navigation Guard

The `beforeEnter` navigation guard in Vue Router is a route-specific guard that runs before entering a particular route. It allows you to add custom logic for specific routes, such as authentication checks or data pre-fetching.

Src/router/routes.js:

```
import { createRouter, createWebHistory } from 'vue-router'
import HomePage from '@/components/Home/HomePage.vue'
import NotFound from '@/components/Layout/NotFound.vue'
import Login from '@/components/Authentication/Login.vue'
import NoAccess from '@/components/Layout/NoAccess.vue'

function isAdmin() {
  const isAdmin = false
  if (isAdmin) {
    return true
  }
  return { name: 'noAccess' }
}

function isAuthenticated() {
  const isAuthenticated = true
  if (isAuthenticated) {
    return true
  }
  return false
}

const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: '/',
      component: HomePage,
      name: 'home',
    },
    {
      path: '/productList',
      component: ProductList,
      name: 'productList',
      beforeEnter: [isAdmin, isAuthenticated],
    },
    {
      path: '/login',
      component: Login,
      name: 'login',
    },
    {
      path: '/noaccess',
      component: NoAccess,
      name: 'noAccess',
    },
    {
      path: '/*',
      component: NotFound,
    },
  ],
})
```

```

router.beforeEach((to, from) => {
  console.log('Global Before Each')
  const isAuthenticated = true
  if (to.name == 'home') {
    return true
  }
  if (!isAuthenticated && to.name !== 'login') {
    return { name: 'login' }
  }
  return true
})

export default router

```

src/components/Layout/NoAccess.vue:

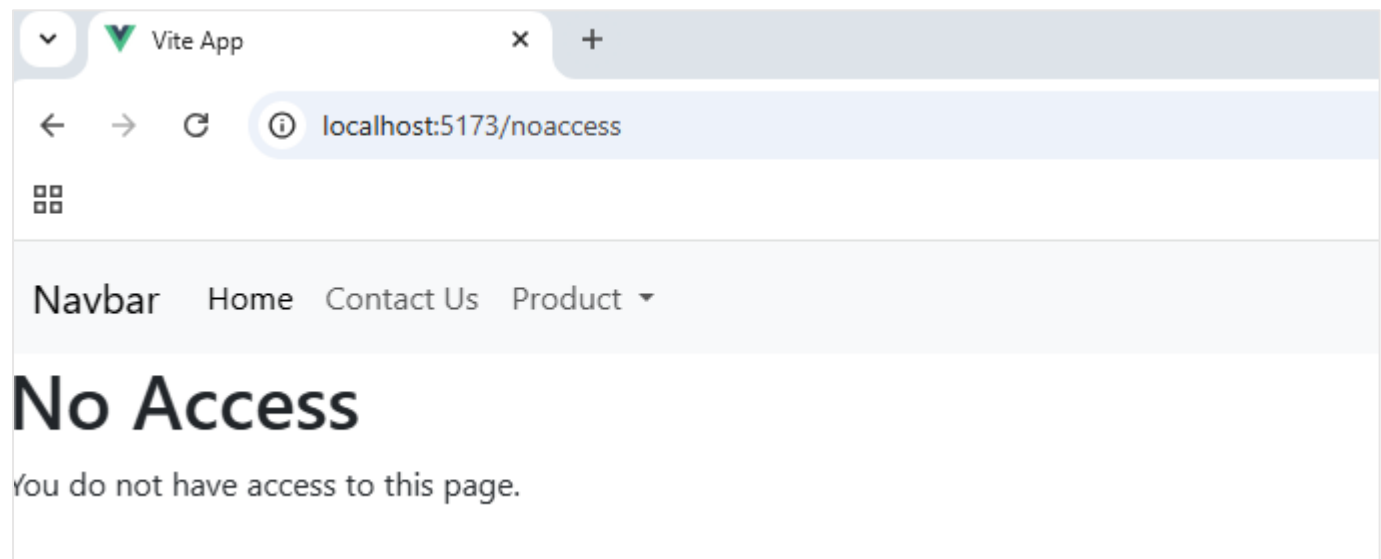
```

<template>
  <div>
    <h1>No Access</h1>
    <p>You do not have access to this page.</p>
  </div>
</template>

```

Output:

If we try to navigate to /productlist, it redirects us to noaccess component.



Project: Axios in action

Axios is a popular JavaScript library used for making HTTP requests. It's lightweight and promise-based.

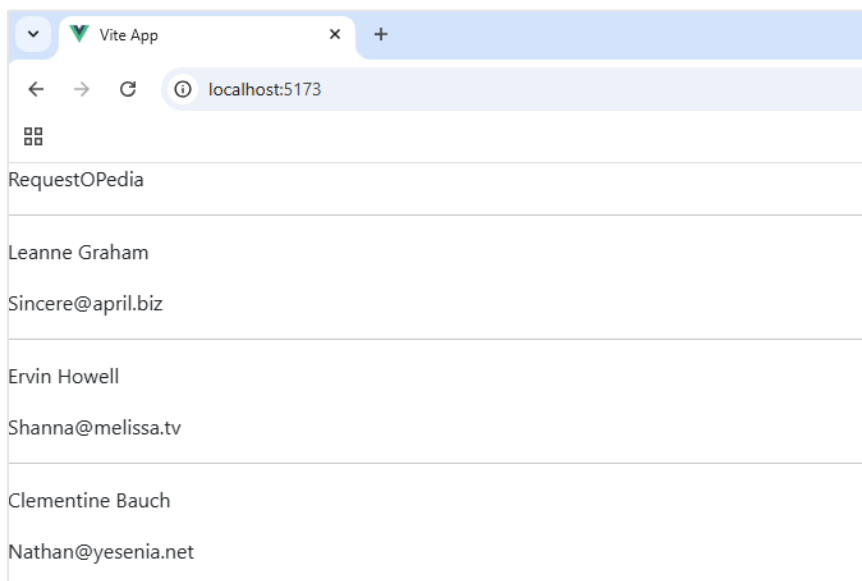
Source Code:

Src/App.vue:

```
<template>
  <div>RequestOPedia</div>
  <hr />
  <div v-for="user in userObj.users" :key="user.id">
    <p>{{ user.name }}</p>
    <p>{{ user.email }}</p>
  <hr />
</div>
</template>
<script setup>
import { reactive, onMounted } from 'vue'
import axios from 'axios'
const userObj = reactive({
  users: [],
})

onMounted(() => {
  axios.get('https://jsonplaceholder.typicode.com/users').then((response) => {
    console.log(response.data)
    userObj.users = response.data
  })
})
</script>
```

Output:



RequestOPedia
Leanne Graham
Sincere@april.biz
Ervin Howell
Shanna@melissa.tv
Clementine Bauch
Nathan@yesenia.net

Project: Fetch in Action

The fetch API is a built-in JavaScript function for making network requests. It provides a modern, promise-based approach to handle HTTP requests and responses.

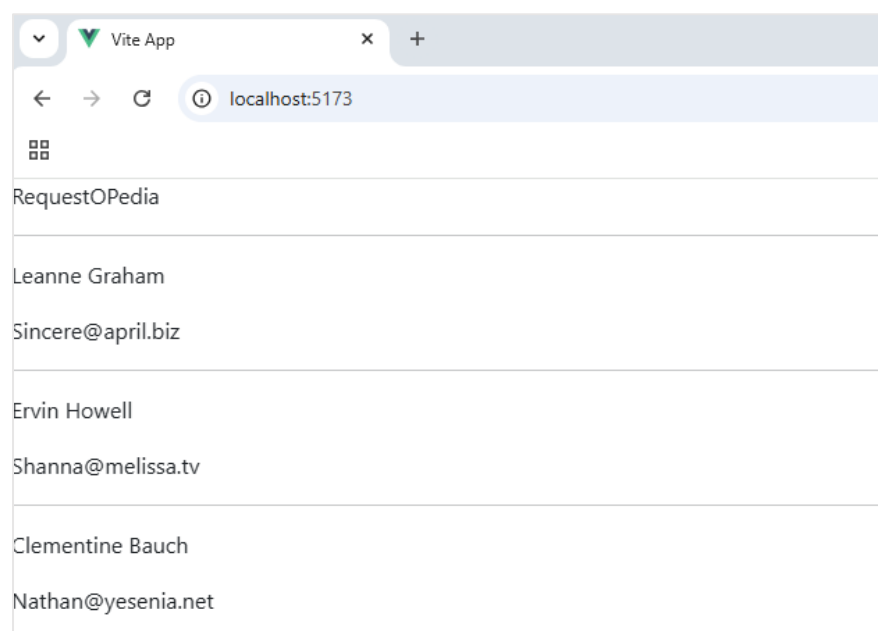
Source Code:

Src/App.vue:

```
<template>
  <div>RequestOPedia</div>
  <hr />
  <div v-for="user in userObj.users" :key="user.id">
    <p>{{ user.name }}</p>
    <p>{{ user.email }}</p>
    <hr />
  </div>
</template>
<script setup>
import { reactive, onMounted } from 'vue'
const userObj = reactive({
  users: [],
})

onMounted(() => {
  fetch('https://jsonplaceholder.typicode.com/users')
    .then((response) => response.json())
    .then((data) => {
      userObj.users = data
    })
})
</script>
```

Output:



RequestOPedia	
Leanne Graham	Sincere@april.biz
Ervin Howell	Shanna@melissa.tv
Clementine Bauch	Nathan@yesenia.net

Project: Pinia

Using Pinia, you can manage state in the application. If there is some data that you want across multiple components you can store that in Pinia Store and then access it from many components in vue application.

Src/main.js //register pinia store to the app

```
import { createApp } from "vue";
import { createPinia } from "pinia";
import App from "./App.vue";
import router from "./router";

const app = createApp(App);

const pinia = createPinia();
app.use(pinia);
app.use(router);

app.mount("#app");
```

src/store/gamestore.js //Create Pinia Store here and add state where we have added 4 properties.

```
import { defineStore } from "pinia";

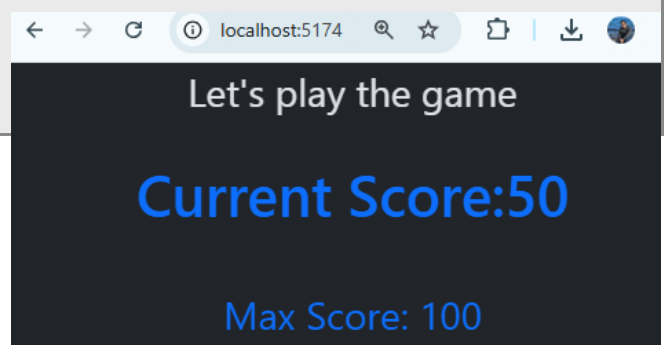
export const useGameStore = defineStore("gameStore", {
  state: () => ({
    score: 50,
    maxHeath: 100,
    maxAttack: 30,
    maxDefense: 10,
  }),
});
```

Src/components/home.vue //Consume store in any component

```
<template>
  <div class="text-center">
    <p>Let's play the game</p>
    <h2 class="text-primary pb-3">Current Score:{{ gameStore.score }}</h2>
    <span class="text-primary pb-3">Max Score: {{ gameStore.maxHeath }}</span>
  </div>
</template>

<script setup>
import { useGameStore } from "../store/gameStore";

const gameStore = useGameStore();
</script>
```



Project: Pinia Actions and Getters

You can add getters in pinia store to fetch some data with some calculations, and you can use actions which are basically functions in pinia store to perform some operations. Actions can also change the state of pinia store.

In the example below you can see the store is shared across home and info component

Source Code:

Src/store/gamestore.js

```
import { defineStore } from "pinia";

export const useGameStore = defineStore("gameStore", {
  state: () => ({
    score: 50,
    maxHeath: 100,
    maxAttack: 30,
    maxDefense: 10,
  }),
  getters: {
    getScore() {
      return this.score;
    },
    getWinningScore() {
      return this.maxHeath;
    },
  },
  actions: {
    setNextAttack() {
      let attack = Math.floor(Math.random() * this.maxAttack) + 1;
      console.log("attack: ", attack);
      this.score += attack;
    },
    setNextDefense() {
      let defense = Math.floor(Math.random() * this.maxDefense) + 1;
      console.log("defense: ", defense);
      this.score -= defense;
    },
    resetScore() {
      this.score = 50;
    },
  },
});
```

Src/components/home.vue

```
<template>
  <div class="text-center">
    <p>Let's play the game</p>
    <h2 class="text-primary pb-3">Current Score:{{ gameStore.getScore }}</h2>
    <span class="text-primary pb-3">Max Score: {{ gameStore.maxHeath }}</span>
    <br />
    <span
      class="text-success pb-3 h3"
      v-if="gameStore.getScore >= gameStore.maxHeath"
    >
```

```

    You Won!
  </span>
  <span class="text-danger pb-3 h3" v-else-if="gameStore.getScore < 0">
    You Lost!
  </span>
  <div
    class="row"
    v-if="gameStore.getScore < gameStore.maxHeath && gameStore.getScore > 0"
  >
    <div class="col-5 offset-1">
      <button class="form-control btn btn-success p-4" @click="Increment"> Increment </button>
    </div>
    <div class="col-5">
      <button class="form-control btn btn-danger p-4" @click="Decrement"> Decrement </button>
    </div>
    <div class="col-6 offset-3 pt-2">
      <button class="form-control btn btn-warning p-4" @click="Random"> Random </button>
    </div>
  </div>
  <div v-else>
    <button
      class="form-control btn btn-primary p-4"
      @click="gameStore.resetScore()"
    >
      Reset Game
    </button>
  </div>
</div>
</template>
<script setup>
import { useGameStore } from "../store/gameStore";

function Increment() {
  gameStore.setNextAttack();
}
function Decrement() {
  gameStore.setNextDefense();
}
function Random() {
  Math.random() > 0.5 ? Increment() : Decrement();
}
const gameStore = useGameStore();
</script>

```

Src/components/info.vue

```

<template>
  <div>
    <h1 class="text-center">Info</h1>
    <p class="text-center">This is the Info page</p>
    <p>
      This is a simple game where you can increment, decrement, or randomize the
      score. The score starts at 50 and you can win by reaching
      {{ gameStore.getWinningScore }}.
    </p>

    Winning Score:

```

```

    <input type="text" class="form-control" v-model="gameStore.maxHeath" />
  </div>
</template>

<script setup>
import { useGameStore } from "../store/gameStore";
const gameStore = useGameStore();
</script>

```

Src/router/index.js

```

import { createRouter, createWebHistory } from "vue-router";
import Home from "@components/Home.vue";
import Info from "@components/Info.vue";
const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: "/", name: "Home", component: Home,
    },
    {
      path: "/info", name: "info", component: Info,
    },
  ],
});
export default router;

```

src/app.vue

```

<template>
<div class="text-center pt-3 text-success h3">CountOPedia</div>
<hr />
<div class="row">
  <div class="col-4 offset-1">
    <router-link
      class="form-control badge rounded-pill text-bg-warning p-4"
      aria-current="page"
      to="/"
    >Home</router-link>
  </div>
  <div class="col-4 offset-1">
    <router-link
      class="form-control badge rounded-pill text-bg-warning p-4"
      aria-current="page"
      to="/info"
    >Info</router-link>
  </div>
</div>
<RouterView />
</template>
<script setup>
import { RouterLink, RouterView } from "vue-router";
</script>

```


Output:

